

2026-05-05-p1-f02-permission-rules

P1-F02 — Permission Rule System Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Port claude-code's 5-mode permission rule system to HelixCode by extending the existing `internal/tools/confirmation/PolicyEngine` with wildcard-pattern matching, layered file-based rule loading, smuggle-resistant compound-command splitting, and a full CLI/slash-command surface.

Architecture: New thin package `internal/tools/permissions/` produces `confirmation.Policy` objects from YAML files (user + project) and built-in mode presets. Existing `confirmation.Condition` gets one new field (`Wildcard string`). `mvdan.cc/sh/v3/syntax` walks Bash inputs to extract every leaf call expression — compound commands aggregate to most-restrictive (deny > ask > allow). CLI exposes --
`permission-mode flag + helixcode permissions {list,add,remove,check}`
Cobra group + `/permissions slash command` via existing `internal/commands/`.

Tech Stack: Go 1.26, testify v1.11, `gopkg.in/yaml.v3`, `github.com/spf13/cobra v1.8`, **NEW** `mvdan.cc/sh/v3` (added in T04), existing `internal/tools/confirmation`, existing `internal/commands`.

Spec: `docs/superpowers/specs/2026-05-05-p1-f02-permission-rules-design.md` (commit `f9e97ff`)

Working directory for all go commands: `helix_code/` (the inner Go module).

Anti-bluff smoke (run on every commit):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/tools/permissions/ && echo "BLUFF FOUND" || echo "clean"
```

Task 1: Bootstrap evidence + advance PROGRESS

Files: - Modify: `docs/improvements/06_phase_1_evidence.md` - Modify: `docs/improvements/PROGRESS.md`



Step 1: Append F02 section header to evidence file

Append to `docs/improvements/06_phase_1_evidence.md`:

P1-F02 — Permission Rule System

****Spec:**** ``docs/superpowers/specs/2026-05-05-p1-f02-permission-rules-design.md` (commit `f9e97ff`)`
****Plan:**** ``docs/superpowers/plans/2026-05-05-p1-f02-permission-rules.md``
****Started:**** 2026-05-05
****Status:**** active

Task evidence trail

(filled in commit-by-commit as tasks land)



Step 2: Update PROGRESS.md current focus

Edit docs/improvements/PROGRESS.md lines 10-18 (the “Current focus” block):

Current focus

- ****Active phase:**** P1 — claude-code feature porting
- ****Active feature:**** F02 — Permission Rule System
- ****Active task:**** P1-F02-T01 — bootstrap evidence + advance PROGRESS
- ****Last completed:**** P1-F01-T11 — Feature 1 (Auto-Compaction) close-out + push
- ****Owner:**** agent (Claude Opus 4.7)
- ****Started:**** 2026-05-04
- ****Last touched:**** 2026-05-05
- ****Blocked-on:**** none



Step 3: Add F02 task list to PROGRESS.md

After the F01 task list block, insert a new block:

Active feature task list (P1-F02: Permission Rule System)

- [] P1-F02-T01 — bootstrap evidence + advance PROGRESS
- [] P1-F02-T02 — add Wildcard field to confirmation.Condition (TDD)
- [] P1-F02-T03 — internal/tools/permissions package skeleton
- [] P1-F02-T04 — shell_splitter.go + mvdan.cc/sh/v3 dep (TDD)
- [] P1-F02-T05 — rule_engine.go pattern parse + match + priority (TDD)
- [] P1-F02-T06 — mode_presets.go five presets + command lists (TDD)
- [] P1-F02-T07 — rule_loader.go YAML + file precedence (TDD)

- [] P1-F02-T08 – permissions.go facade + PolicyEngine registration
- [] P1-F02-T09 – wire --permission-mode flag + integration test (no mocks)
- [] P1-F02-T10 – helixcode permissions {list,add,remove,check} subcommands
- [] P1-F02-T11 – /permissions slash command via internal/commands
- [] P1-F02-T12 – Challenge with three runtime-evidence scenarios
- [] P1-F02-T13 – Feature 2 close-out + push



Step 4: Commit

```
git add docs/improvements/06_phase_1_evidence.md docs/
improvements/PROGRESS.md
git commit -m "$(cat <<'EOF'
docs(P1-F02-T01): bootstrap Phase 1 / Feature 2 evidence +
advance PROGRESS
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 2: Add Wildcard field to confirmation.Condition (TDD)

Files: - Modify: helix_code/internal/tools/confirmation/policies.go (the Condition struct + Matches) - Test: helix_code/internal/tools/confirmation/policies_wildcard_test.go (NEW)



Step 1: Write failing test

Create helix_code/internal/tools/confirmation/policies_wildcard_test.go:

```
package confirmation

import (
    "testing"

    "github.com/stretchr/testify/assert"
)

func TestCondition_Wildcard_MatchesGlob(t *testing.T) {
    cond := Condition{
```

```

        ToolName: "Bash",
        Wildcard: "git status*",
    }
    req := ConfirmationRequest{
        ToolName: "Bash",
        Parameters: map[string]interface{}{"command":
            "git status -sb"},
    }
    assert.True(t, cond.Matches(req))
}

func TestCondition_Wildcard_DoesNotMatch(t *testing.T) {
    cond := Condition{
        ToolName: "Bash",
        Wildcard: "git status*",
    }
    req := ConfirmationRequest{
        ToolName: "Bash",
        Parameters: map[string]interface{}{"command": "git push
            origin main"},
    }
    assert.False(t, cond.Matches(req))
}

func TestCondition_Wildcard_QuestionMark(t *testing.T) {
    cond := Condition{
        ToolName: "Read",
        Wildcard: "? .go",
    }
    req := ConfirmationRequest{
        ToolName: "Read",
        Parameters: map[string]interface{}{"path": "x.go"},
    }
    assert.True(t, cond.Matches(req))
}

func TestCondition_Wildcard_EmptyMatchesAll(t *testing.T) {
    cond := Condition{
        ToolName: "Read",
        Wildcard: "",
    }
    req := ConfirmationRequest{
        ToolName: "Read",
        Parameters: map[string]interface{}{"path":
            "anything.txt"},
    }
    assert.True(t, cond.Matches(req))
}

```

```
func TestCondition_Wildcard_MissingParameterIsNoMatch(t
    *testing.T) {
    cond := Condition{
        ToolName: "Bash",
        Wildcard: "git status*",
    }
    req := ConfirmationRequest{
        ToolName: "Bash",
        Parameters: map[string]interface{}{},
    }
    assert.False(t, cond.Matches(req))
}
```



Step 2: Run test to verify it fails

```
cd HelixCode && go test -count=1 -run TestCondition_Wildcard ./
    internal/tools/confirmation/
```

Expected: FAIL — Condition has no Wildcard field.



Step 3: Add Wildcard field + matching logic

In `helix_code/internal/tools/confirmation/policies.go`, modify the `Condition` struct (around line 30-36):

```
// Condition defines matching criteria
type Condition struct {
    ToolName      string
    OperationType []OperationType
    RiskLevel     []RiskLevel
    PathPattern   string
    Wildcard       string           // NEW: glob
                  pattern matched against the request's primary string
                  parameter
    Custom        func(ConfirmationRequest) bool
}
```

Add a helper at the bottom of `policies.go`:

```
// wildcardMatch implements glob-style matching with *, ?, and
    [abc] character classes.
// Returns true if pattern is empty (matches all).
func wildcardMatch(pattern, s string) bool {
    if pattern == "" {
        return true
    }
    return globMatch(pattern, s)
```

```

}

// globMatch is a non-regex glob matcher that handles *, ?, and
// [abc].
func globMatch(pattern, s string) bool {
    // Iterative glob matcher: walk pattern and string in lock-
    // step.
    pi, si := 0, 0
    starPi, starSi := -1, 0
    for si < len(s) {
        if pi < len(pattern) && pattern[pi] == '\\' && pi+1 <
            len(pattern) {
            if pattern[pi+1] == s[si] {
                pi += 2
                si++
                continue
            }
        } else if pi < len(pattern) && pattern[pi] == '?' {
            pi++
            si++
            continue
        } else if pi < len(pattern) && pattern[pi] == '[' {
            closeIdx := pi + 1
            for closeIdx < len(pattern) && pattern[closeIdx] !=
                ']' {
                closeIdx++
            }
            if closeIdx >= len(pattern) {
                return false
            }
            class := pattern[pi+1 : closeIdx]
            matched := false
            for _, c := range []byte(class) {
                if c == s[si] {
                    matched = true
                    break
                }
            }
            if matched {
                pi = closeIdx + 1
                si++
                continue
            }
        } else if pi < len(pattern) && pattern[pi] == '*' {
            starPi = pi
            starSi = si
            pi++
            continue
        }
    }
}

```

```

    } else if pi < len(pattern) && pattern[pi] == s[si] {
        pi++
        si++
        continue
    }
    if starPi != -1 {
        pi = starPi + 1
        starSi++
        si = starSi
        continue
    }
    return false
}
for pi < len(pattern) && pattern[pi] == '*' {
    pi++
}
return pi == len(pattern)
}

```

Modify `Condition.Matches` (around line 38-86) to evaluate Wildcard after the existing checks. Insert this block immediately before the `// Custom condition` block:

```

// Match wildcard against primary string parameter
if c.Wildcard != "" {
    primary := primaryStringParam(req)
    if !wildcardMatch(c.Wildcard, primary) {
        return false
    }
}

```

Add the helper function `primaryStringParam` at the bottom of `policies.go`:

```

// primaryStringParam returns the most-relevant string parameter
// for wildcard matching.
// For Bash: req.Parameters["command"]. For file tools:
// req.Parameters["path"] or req.Operation.Target.
func primaryStringParam(req ConfirmationRequest) string {
    if cmd, ok := req.Parameters["command"].(string); ok {
        return cmd
    }
    if p, ok := req.Parameters["path"].(string); ok {
        return p
    }
    if p, ok := req.Parameters["file_path"].(string); ok {
        return p
    }
}

```

```
    return req.Operation.Target
}
```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -race -run
    TestCondition_Wildcard ./internal/tools/confirmation/
```

Expected: PASS (5 tests).



Step 5: Run the full confirmation package to verify no regression

```
cd HelixCode && go test -count=1 -race ./internal/tools/
confirmation/...
```

Expected: PASS (all existing tests still green).



Step 6: Commit

```
git -C HelixCode add internal/tools/confirmation/policies.go
    internal/tools/confirmation/policies_wildcard_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T02): add Wildcard field to confirmation.Condition
```

```
TDD: 5 unit tests covering glob matching, character classes,
    empty
pattern, and missing parameter. Adds primaryStringParam helper to
extract the matchable string from the request. No regression in
existing PolicyEngine tests.
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
    <noreply@anthropic.com>
```

```
EOF
)"
```

Task 3: Permissions package skeleton

Files: - Create: helix_code/internal/tools/permissions/doc.go - Create:
helix_code/internal/tools/permissions/types.go



Step 1: Create doc.go

Create helix_code/internal/tools/permissions/doc.go:


```
// Package permissions implements the claude-code-style
// permission rule system.
//
// It is a thin layer that loads YAML rule files (user +
// project), composes them
// with one of five built-in mode presets (default, auto,
// acceptEdits, dontAsk,
// bypassPermissions), and produces a confirmation.Policy that
// registers with
// the existing confirmation.PolicyEngine. Compound Bash commands
// are split via
// mvdan.cc/sh/v3/syntax so command substitutions, backticks,
// heredocs, and
// pipelines all aggregate to most-restrictive (deny > ask >
// allow).
//
// See: docs/superpowers/specs/2026-05-05-p1-f02-permission-
// rules-design.md
package permissions
```



Step 2: Create types.go

Create `helix_code/internal/tools/permissions/types.go`:

```
package permissions

import (
    "dev.helix.code/internal/tools/confirmation"
)

// Scope identifies where a rule lives.
type Scope int

const (
    ScopeCLI Scope = iota
    ScopeProject
    ScopeUser
    ScopePreset
)

// String returns the lowercase scope name.
func (s Scope) String() string {
    switch s {
    case ScopeCLI:
        return "cli"
    case ScopeProject:
        return "project"
    case ScopeUser:
```

```

        return "user"
    case ScopePreset:
        return "preset"
    default:
        return "unknown"
    }
}

// Rule is a single permission rule.
type Rule struct {
    Pattern      string           // e.g. "Bash(git status:*)"
    Action       confirmation.Action // ActionAllow / ActionAsk /
        ActionDeny
    Priority      int               // higher wins; default 0
    Description   string
    Source
        Scope           // where this rule was loaded from
}

// RuleSet is the resolved set of rules for a session.
type RuleSet struct {
    Mode         string // "default" | "auto" | "acceptEdits" |
        "dontAsk" | "bypassPermissions"
    Rules        []Rule
    Sources      []string // file paths in load order
}

// ValidModes lists the five preset names accepted by --
// permission-mode and the YAML mode: key.
var ValidModes = []string{"default", "auto", "acceptEdits",
    "dontAsk", "bypassPermissions"}

// IsValidMode reports whether name is a recognised preset.
func IsValidMode(name string) bool {
    for _, m := range ValidModes {
        if m == name {
            return true
        }
    }
    return false
}

```



Step 3: Verify the package compiles

```
cd HelixCode && go build ./internal/tools/permissions/...
```

Expected: clean compile, exit 0.



Step 4: Run anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/tools/permissions/ && echo "BLUFF
FOUND" || echo "clean"
```

Expected: clean.



Step 5: Commit

```
git -C HelixCode add internal/tools/permissions/doc.go internal/
tools/permissions/types.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T03): add internal/tools/permissions package skeleton
```

```
Doc.go declares package purpose. Types.go defines Scope, Rule,
RuleSet,
ValidModes, and IsValidMode. Compiles clean; anti-bluff smoke
clean.
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
```

```
EOF
)"
```

Task 4: Shell splitter with mvdan.cc/sh (TDD)

Files: - Create: helix_code/internal/tools/permissions/
 shell_splitter.go - Test: helix_code/internal/tools/permissions/
 shell_splitter_test.go - Modify: helix_code/go.mod + helix_code/go.sum
 (auto-generated)



Step 1: Write failing test

Create helix_code/internal/tools/permissions/shell_splitter_test.go:

```
package permissions

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestSplitCommands_SingleCommand(t *testing.T) {
```

```

    cmds, err := SplitCommands("git status -sb")
    require.NoError(t, err)
    assert.Equal(t, []string{"git status -sb"}, cmds)
}

func TestSplitCommands_AndChain(t *testing.T) {
    cmds, err := SplitCommands("ls && git push")
    require.NoError(t, err)
    assert.Equal(t, []string{"ls", "git push"}, cmds)
}

func TestSplitCommands_OrChainAndSemicolon(t *testing.T) {
    cmds, err := SplitCommands("ls || cat readme; rm tmp")
    require.NoError(t, err)
    assert.Equal(t, []string{"ls", "cat readme", "rm tmp"}, cmds)
}

func TestSplitCommands_Pipeline(t *testing.T) {
    cmds, err := SplitCommands("cat foo | grep bar | wc -l")
    require.NoError(t, err)
    assert.Equal(t, []string{"cat foo", "grep bar", "wc -l"},
        cmds)
}

func TestSplitCommands_CommandSubstitution_DollarParen(t
    *testing.T) {
    cmds, err := SplitCommands("echo $(rm -rf /tmp/x)")
    require.NoError(t, err)
    assert.Contains(t, cmds, "rm -rf /tmp/x")
    assert.Contains(t, cmds, "echo")
}

func TestSplitCommands_CommandSubstitution_Backticks(t
    *testing.T) {
    cmds, err := SplitCommands("echo `rm -rf /tmp/x`")
    require.NoError(t, err)
    assert.Contains(t, cmds, "rm -rf /tmp/x")
}

func TestSplitCommands_QuotedOperatorIsLiteral(t *testing.T) {
    cmds, err := SplitCommands(`echo "foo && bar"`)
    require.NoError(t, err)
    assert.Equal(t, 1, len(cmds), "quoted && must NOT be split")
    assert.Contains(t, cmds[0], `foo && bar`)
}

func TestSplitCommands_Heredoc(t *testing.T) {
    input := "cat <<EOF\nhello\nEOF\nrm /tmp/x"

```

```

    cmds, err := SplitCommands(input)
    require.NoError(t, err)
    assert.Contains(t, cmds, "rm /tmp/x")
}

func TestSplitCommands_MalformedReturnsError(t *testing.T) {
    _, err := SplitCommands(`echo "unclosed"`)
    require.Error(t, err)
}

func TestSplitCommands_EmptyInput(t *testing.T) {
    cmds, err := SplitCommands("")
    require.NoError(t, err)
    assert.Empty(t, cmds)
}

```



Step 2: Add the dependency

```
cd HelixCode && go get mvdan.cc/sh/v3@latest
```

Expected: dep added; go.mod and go.sum updated.



Step 3: Run test to verify it fails

```
cd HelixCode && go test -count=1 -run TestSplitCommands ./
internal/tools/permissions/
```

Expected: FAIL — SplitCommands undefined.



Step 4: Implement SplitCommands

Create helix_code/internal/tools/permissions/shell_splitter.go:

```
package permissions
```

```
import (
    "strings"
```

```
    "mvdan.cc/sh/v3/syntax"
```

```
)
```

```
// SplitCommands extracts every leaf call expression from a shell
    script,
```

```
// recursing into command substitutions (${...}), backticks).
```

```
    Returns one
```

```
// entry per leaf command.
```

```
//
```

```

// Quoted operators ("foo && bar") are NOT split – the parser
// treats them
// as literal arguments. A malformed input returns a non-nil
// error so that
// callers can fail-closed.
func SplitCommands(input string) ([]string, error) {
    if strings.TrimSpace(input) == "" {
        return nil, nil
    }
    parser := syntax.NewParser()
    f, err := parser.Parse(strings.NewReader(input), "")
    if err != nil {
        return nil, err
    }
    var cmds []string
    syntax.Walk(f, func(n syntax.Node) bool {
        if call, ok := n.(*syntax.CallExpr); ok &&
            len(call.Args) > 0 {
            cmds = append(cmds, callExprString(call))
        }
        return true
    })
    return cmds, nil
}

// callExprString renders a CallExpr to its source-equivalent
// string,
// stripping outer quotes/backticks from word parts so wildcard
// patterns
// match the underlying command text.
func callExprString(call *syntax.CallExpr) string {
    var sb strings.Builder
    for i, arg := range call.Args {
        if i > 0 {
            sb.WriteByte(' ')
        }
        sb.WriteString(wordString(arg))
    }
    return sb.String()
}

// wordString prints a single word; double-quoted parts have
// their quotes
// stripped (the contents are emitted as-is) so that "git push"
// inside
// quotes still produces "git push" for matching.
func wordString(w *syntax.Word) string {
    var sb strings.Builder

```

```

for _, part := range w.Parts {
    switch p := part.(type) {
    case *syntax.Lit:
        sb.WriteString(p.Value)
    case *syntax.SglQuoted:
        sb.WriteString(p.Value)
    case *syntax.DblQuoted:
        for _, inner := range p.Parts {
            if lit, ok := inner.(*syntax.Lit); ok {
                sb.WriteString(lit.Value)
            }
        }
    case *syntax.CmdSubst:
        // Substitutions render as their inner text; the leaf
        // walk in
        // SplitCommands has already extracted them as
        // separate calls.
        // Emit a placeholder so the parent's wildcard match
        // still sees
        // the surrounding tokens correctly.
        sb.WriteString("$(")
        for _, stmt := range p.Stmts {
            if call, ok := stmt.Cmd.(*syntax.CallExpr); ok {
                sb.WriteString(callExprString(call))
            }
        }
        sb.WriteString(")")
    default:
        sb.WriteString(syntax.NodeString(part))
    }
}
return sb.String()
}

```



Step 5: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -race -run TestSplitCommands ./
    internal/tools/permissions/

```

Expected: PASS (10 tests).



Step 6: Run anti-bluff smoke

```

cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
    placeholder" internal/tools/permissions/ && echo "BLUFF
    FOUND" || echo "clean"

```

Expected: clean.



Step 7: Commit

git

```
-C HelixCode add internal/tools/permissions/  
shell_splitter.go internal/tools/permissions/  
shell_splitter_test.go go.mod go.sum
```

```
git commit -m "$(cat <<'EOF'
```

```
feat(P1-F02-T04): smuggle-resistant shell splitter via mvdan.cc/  
sh/v3
```

10 unit tests covering &&, ||, ;, |, \$(...), backticks, heredocs, quoted-operator literals, malformed input (error), and empty input.

The walker descends into CmdSubst so command substitutions emit their

inner calls as separate leaves – closing the smuggling vector that

the porting doc's simple splitter leaves open.

Co-Authored-By: Claude Opus 4.7 (1M context)

<noreply@anthropic.com>

EOF

)"

Task 5: Rule engine — pattern parse, match, priority sort (TDD)

Files: - Create: helix_code/internal/tools/permissions/rule_engine.go -

Test: helix_code/internal/tools/permissions/rule_engine_test.go



Step 1: Write failing test

Create helix_code/internal/tools/permissions/rule_engine_test.go:

```
package permissions
```

```
import (
```

```
    "testing"
```

```
    "github.com/stretchr/testify/assert"
```

```
    "github.com/stretchr/testify/require"
```

```
    "dev.helix.code/internal/tools/confirmation"
```

```
)
```



```

func TestParsePattern_Valid(t *testing.T) {
    tests := []struct {
        in      string
        toolName string
        argPat   string
    }{
        {"Bash(git status:*)", "Bash", "git status:*"},
        {"Read(*.go)", "Read", "*.go"},
        {"Edit(internal/auth/*)", "Edit", "internal/auth/*"},
        {"Write()", "Write", ""},
        {"*(*)", "*", "*"},
    }
    for _, tt := range tests {
        t.Run(tt.in, func(t *testing.T) {
            got, err := ParsePattern(tt.in)
            require.NoError(t, err)
            assert.Equal(t, tt.toolName, got.ToolName)
            assert.Equal(t, tt.argPat, got.ArgPattern)
        })
    }
}

func TestParsePattern_Malformed(t *testing.T) {
    bad := []string{"Bash", "Bash(", "Bash)", "(foo)", "",
        "Bash()(extra)"}
    for _, b := range bad {
        t.Run(b, func(t *testing.T) {
            _, err := ParsePattern(b)
            assert.Error(t, err)
        })
    }
}

func TestEvaluate_PriorityOrder(t *testing.T) {
    rules := []Rule{
        {Pattern: "Bash(git push*)", Action:
            confirmation.ActionDeny, Priority: 1000},
        {Pattern: "Bash(*)", Action: confirmation.ActionAllow,
            Priority: 1},
    }
    eng, err := NewRuleEngine(rules)
    require.NoError(t, err)
    got := eng.Evaluate("Bash", "git push origin main")
    assert.Equal(t, confirmation.ActionDeny, got.Action)
    assert.Equal(t, "Bash(git push*)", got.MatchedPattern)
}

```

```

func TestEvaluate_NoMatchReturnsAsk(t *testing.T) {
    rules := []Rule{
        {Pattern: "Bash(ls*)", Action: confirmation.ActionAllow},
    }
    eng, err := NewRuleEngine(rules)
    require.NoError(t, err)
    got := eng.Evaluate("Read", "/etc/passwd")
    assert.Equal(t, confirmation.ActionAsk, got.Action)
    assert.Empty(t, got.MatchedPattern)
}

```

```

func TestEvaluate_CompoundAggregation_DenyWins(t *testing.T) {
    rules := []Rule{
        {Pattern: "Bash(echo*)", Action:
            confirmation.ActionAllow},
        {Pattern: "Bash(rm*)", Action: confirmation.ActionDeny,
            Priority: 100},
    }
    eng, err := NewRuleEngine(rules)
    require.NoError(t, err)
    got := eng.Evaluate("Bash", "echo hello && rm -rf /tmp/x")
    assert.Equal(t, confirmation.ActionDeny, got.Action)
}

```

```

func TestEvaluate_CompoundAggregation_AskOverAllow(t *testing.T)
{
    rules := []Rule{
        {Pattern: "Bash(echo*)", Action:
            confirmation.ActionAllow},
        {Pattern: "Bash(curl*)", Action: confirmation.ActionAsk},
    }
    eng, err := NewRuleEngine(rules)
    require.NoError(t, err)
    got := eng.Evaluate("Bash", "echo hi && curl example.com")
    assert.Equal(t, confirmation.ActionAsk, got.Action)
}

```

```

func TestEvaluate_CommandSubstitutionIsAggregated(t *testing.T) {
    rules := []Rule{
        {Pattern: "Bash(echo*)", Action:
            confirmation.ActionAllow},
        {Pattern: "Bash(rm*)", Action: confirmation.ActionDeny,
            Priority: 100},
    }
    eng, err := NewRuleEngine(rules)
    require.NoError(t, err)
    got := eng.Evaluate("Bash", "echo $(rm -rf /tmp/x)")
    assert.Equal(t, confirmation.ActionDeny, got.Action,

```

```

        "smuggled rm inside $() must propagate deny to compound")
    }

func TestEvaluate_ShellParseErrorIsDeny(t *testing.T) {
    rules := []Rule{
        {Pattern: "Bash(*)", Action: confirmation.ActionAllow},
    }
    eng, err := NewRuleEngine(rules)
    require.NoError(t, err)
    got := eng.Evaluate("Bash", `echo "unclosed"`)
    assert.Equal(t, confirmation.ActionDeny, got.Action,
        "shell parse error must fail-closed to deny")
}

func TestNewRuleEngine_RejectsMalformedPatterns(t *testing.T) {
    rules := []Rule{
        {Pattern: "Bash(", Action: confirmation.ActionAllow},
    }
    _, err := NewRuleEngine(rules)
    require.Error(t, err)
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run 'TestParsePattern|
    TestEvaluate|TestNewRuleEngine' ./internal/tools/
    permissions/

```

Expected: FAIL — ParsePattern, NewRuleEngine, etc. undefined.



Step 3: Implement rule_engine.go

Create helix_code/internal/tools/permissions/rule_engine.go:

```

package permissions

import (
    "fmt"
    "regexp"
    "sort"
    "strings"

    "dev.helix.code/internal/tools/confirmation"
)

// ParsedPattern is a decomposed permission rule pattern.
type ParsedPattern struct {
    ToolName    string

```

```

    ArgPattern string
}

// Decision is the outcome of evaluating a request against the
// rule engine.
type Decision struct {
    Action          confirmation.Action
    MatchedPattern  string // empty if no rule matched
    Reason          string // human-readable explanation for audit logs
}

// patternRegex parses "ToolName(arg-pattern)" with optional
// whitespace.
var patternRegex = regexp.MustCompile(`^([A-Za-z0-9_\\*]+)\\s*\\
    ((\\^())*)\\)$`)

// ParsePattern decomposes "ToolName(arg)" into its parts.
// Returns an error for malformed input.
func ParsePattern(pattern string) (ParsedPattern, error) {
    pattern = strings.TrimSpace(pattern)
    if pattern == "" {
        return ParsedPattern{}, fmt.Errorf("empty pattern")
    }
    m := patternRegex.FindStringSubmatch(pattern)
    if m == nil {
        return ParsedPattern{}, fmt.Errorf("malformed pattern %q
        (expected ToolName(arg-pattern))", pattern)
    }
    return ParsedPattern{ToolName: m[1], ArgPattern: m[2]}, nil
}

// RuleEngine holds parsed rules sorted by priority.
type RuleEngine struct {
    rules []parsedRule
}

type parsedRule struct {
    parsed ParsedPattern
    rule   Rule
}

// NewRuleEngine parses every rule pattern and sorts by
// descending priority.
// Returns an error if any pattern is malformed (fail-fast at
// load time).
func NewRuleEngine(rules []Rule) (*RuleEngine, error) {
    out := make([]parsedRule, 0, len(rules))

```

```

    for _, r := range rules {
        p, err := ParsePattern(r.Pattern)
        if err != nil {
            return nil, fmt.Errorf("rule %q: %w", r.Pattern, err)
        }
        out = append(out, parsedRule{parsed: p, rule: r})
    }
    sort.SliceStable(out, func(i, j int) bool {
        return out[i].rule.Priority > out[j].rule.Priority
    })
    return &RuleEngine{rules: out}, nil
}

```

// Evaluate resolves a tool call to a single Decision. For Bash, the input is split into leaf calls and aggregated to the most-restrictive (deny > ask >

// allow). A shell parse error is treated as deny (fail-closed).

```

func (e *RuleEngine) Evaluate(toolName, input string) Decision {
    if toolName == "Bash" {
        leaves, err := SplitCommands(input)
        if err != nil {
            return Decision{
                Action: confirmation.ActionDeny,
                Reason: fmt.Sprintf("shell parse error: %v",
err),
            }
        }
        if len(leaves) == 0 {
            leaves = []string{input}
        }
        return e.aggregate(toolName, leaves)
    }
    return e.evaluateLeaf(toolName, input)
}

```

```

func (e *RuleEngine) evaluateLeaf(toolName, leaf string)
Decision {
    for _, pr := range e.rules {
        if !toolNameMatches(pr.parsed.ToolName, toolName) {
            continue
        }
        if !wildcardMatchString(pr.parsed.ArgPattern, leaf) {
            continue
        }
        return Decision{
            Action:          pr.rule.Action,
            MatchedPattern: pr.rule.Pattern,

```

```

        Reason:          fmt.Sprintf("matched rule %q (%s)",
pr.rule.Pattern, pr.rule.Source),
    }
}
return Decision{Action: confirmation.ActionAsk, Reason: "no
rule matched"}
}

func (e *RuleEngine) aggregate(toolName string, leaves []string)
Decision {
    worst := confirmation.ActionAllow
    var matched string
    var reason string
    hadDeny, hadAsk := false, false
    for _, leaf := range leaves {
        d := e.evaluateLeaf(toolName, leaf)
        switch d.Action {
        case confirmation.ActionDeny:
            if !hadDeny {
                matched = d.MatchedPattern
                reason = d.Reason
            }
            hadDeny = true
        case confirmation.ActionAsk:
            if !hadDeny && !hadAsk {
                matched = d.MatchedPattern
                reason = d.Reason
            }
            hadAsk = true
        case confirmation.ActionAllow:
            if !hadDeny && !hadAsk && matched == "" {
                matched = d.MatchedPattern
                reason = d.Reason
            }
        }
    }
    switch {
    case hadDeny:
        worst = confirmation.ActionDeny
    case hadAsk:
        worst = confirmation.ActionAsk
    }
    return Decision{Action: worst, MatchedPattern: matched,
Reason: reason}
}

// toolNameMatches treats "*" as a wildcard tool-name match.
func toolNameMatches(pattern, name string) bool {

```

```

    if pattern == "*" {
        return true
    }
    return pattern == name
}

// wildcardMatchString matches the arg-pattern against an input
// string.
// Empty arg-pattern (Tool()) matches only an empty input.
// "*" matches everything.
func wildcardMatchString(pattern, input string) bool {
    if pattern == "" {
        return input == ""
    }
    return globMatchPermissions(pattern, input)
}

// globMatchPermissions is a glob matcher with *, ?, and [abc]
// support.
// Duplicated from confirmation.globMatch to keep the package
// boundary clean.
func globMatchPermissions(pattern, s string) bool {
    pi, si := 0, 0
    starPi, starSi := -1, 0
    for si < len(s) {
        switch {
        case pi < len(pattern) && pattern[pi] == '\\' && pi+1 <
            len(pattern) && pattern[pi+1] == s[si]:
            pi += 2
            si++
        case pi < len(pattern) && pattern[pi] == '?':
            pi++
            si++
        case pi < len(pattern) && pattern[pi] == '[':
            closeIdx := pi + 1
            for closeIdx < len(pattern) && pattern[closeIdx] !=
                ']' {
                closeIdx++
            }
            if closeIdx >= len(pattern) {
                return false
            }
            class := pattern[pi+1 : closeIdx]
            matched := false
            for j := 0; j < len(class); j++ {
                if class[j] == s[si] {
                    matched = true
                    break
                }
            }
        }
    }

```

```

        }
    }
    if matched {
        pi = closeIdx + 1
        si++
    } else if starPi != -1 {
        pi = starPi + 1
        starSi++
        si = starSi
    } else {
        return false
    }
    case pi < len(pattern) && pattern[pi] == '*':
        starPi = pi
        starSi = si
        pi++
    case pi < len(pattern) && pattern[pi] == s[si]:
        pi++
        si++
    case starPi != -1:
        pi = starPi + 1
        starSi++
        si = starSi
    default:
        return false
}
}
for pi < len(pattern) && pattern[pi] == '*' {
    pi++
}
return pi == len(pattern)
}

```



Step 4: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -race -run 'TestParsePattern|
TestEvaluate|TestNewRuleEngine' ./internal/tools/
permissions/

```

Expected: PASS (all 9 subtests + table-driven cases).



Step 5: Run the whole permissions package

```

cd HelixCode && go test -count=1 -race ./internal/tools/
permissions/...

```

Expected: PASS.



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/tools/permissions/ && echo "BLUFF
FOUND" || echo "clean"
```

Expected: clean.



Step 7: Commit

```
git -C HelixCode add internal/tools/permissions/rule_engine.go
internal/tools/permissions/rule_engine_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T05): RuleEngine with pattern parse + priority +
aggregation
```

Implements ParsePattern (regex-based, fail-fast), NewRuleEngine (priority-sorted), and Evaluate. Bash inputs split via SplitCommands

and aggregated most-restrictive (deny > ask > allow). Shell parse errors map to deny. 9 test groups including command-substitution smuggle propagation.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 6: Mode presets (TDD)

Files: - Create: helix_code/internal/tools/permissions/mode_presets.go -
Test: helix_code/internal/tools/permissions/mode_presets_test.go



Step 1: Write failing test

Create helix_code/internal/tools/permissions/mode_presets_test.go:

```
package permissions

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/tools/confirmation"
```

```

)

func TestPresetRules_Default_Empty(t *testing.T) {
    rules := PresetRules("default")
    assert.Empty(t, rules, "default preset has no built-in
        rules")
}

func TestPresetRules_Auto_AllowsEverything(t *testing.T) {
    rules := PresetRules("auto")
    require.Len(t, rules, 1)
    assert.Equal(t, "(*)", rules[0].Pattern)
    assert.Equal(t, confirmation.ActionAllow, rules[0].Action)
    assert.Equal(t, ScopePreset, rules[0].Source)
}

func TestPresetRules_AcceptEdits_AllowsEditAndWrite(t
    *testing.T) {
    rules := PresetRules("acceptEdits")
    patterns := patternsOf(rules)
    assert.Contains(t, patterns, "Edit(*)")
    assert.Contains(t, patterns, "Write(*)")
    assert.Contains(t, patterns, "MultiEdit(*)")
}

func TestPresetRules_DontAsk_AllowsRead(t *testing.T) {
    rules := PresetRules("dontAsk")
    patterns := patternsOf(rules)
    assert.Contains(t, patterns, "Read(*)")
    assert.Contains(t, patterns, "Glob(*)")
    assert.Contains(t, patterns, "Grep(*)")
}

func TestPresetRules_Bypass_HighestPriority(t *testing.T) {
    rules := PresetRules("bypassPermissions")
    require.Len(t, rules, 1)
    assert.Equal(t, "(*)", rules[0].Pattern)
    assert.Equal(t, confirmation.ActionAllow, rules[0].Action)
    assert.Greater(t, rules[0].Priority, 100_000)
}

func TestPresetRules_UnknownReturnsNil(t *testing.T) {
    assert.Nil(t, PresetRules("nonsense"))
}

func TestReadOnlyCommands_Conservative(t *testing.T) {
    assert.True(t, IsReadOnlyCommand("git status"))
    assert.True(t, IsReadOnlyCommand("ls -la"))
}

```

```

    assert.False(t, IsReadOnlyCommand("git push"))
    assert.False(t, IsReadOnlyCommand("rm /tmp/x"))
}

func TestWriteCommands_Conservative(t *testing.T) {
    assert.True(t, IsWriteCommand("git push origin"))
    assert.True(t, IsWriteCommand("rm -rf /tmp/x"))
    assert.False(t, IsWriteCommand("git status"))
    assert.False(t, IsWriteCommand("ls"))
}

func patternsOf(rules []Rule) []string {
    out := make([]string, 0, len(rules))
    for _, r := range rules {
        out = append(out, r.Pattern)
    }
    return out
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run 'TestPresetRules|
    TestReadOnlyCommands|TestWriteCommands' ./internal/tools/
    permissions/

```

Expected: FAIL — PresetRules, IsReadOnlyCommand, IsWriteCommand undefined.



Step 3: Implement mode_presets.go

Create helix_code/internal/tools/permissions/mode_presets.go:

```

package permissions

import (
    "strings"

    "dev.helix.code/internal/tools/confirmation"
)

// readOnlyCommands lists Bash command prefixes that are auto-
// allowed under dontAsk.
// Conservative — anything not listed asks.
var readOnlyCommands = []string{
    "ls", "cat", "find", "grep", "rg", "head", "tail", "wc",
    "ps", "top", "df", "du",
    "env", "uname", "which", "whoami", "date", "lsb_release",
    "echo", "pwd", "stat",
}

```

```

    "git status", "git log", "git diff", "git branch", "git
        show", "git remote",
    "git config --get", "git rev-parse",
    "go version", "node --version", "npm --version", "python --
        version", "python3 --version",
    "rustc --version", "cargo --version", "java -version", "ruby
        --version",
    "docker --version", "kubectl version --client",
}

// writeCommands lists Bash command prefixes that are auto-
// allowed under acceptEdits
// (because acceptEdits is for edit/write operations including
// via shell).
var writeCommands = []string{
    "git add", "git commit", "git push", "git pull", "git
        merge", "git rebase",
    "git checkout", "git reset", "git stash", "git tag",
    "rm", "mv", "cp", "mkdir", "touch", "chmod", "chown", "ln",
    "rmdir",
    "tar", "zip", "unzip", "wget", "curl",
    "npm install", "npm run", "go get", "go mod", "go build",
    "go install",
    "cargo build", "cargo install", "make", "cmake", "docker",
    "kubectl", "helm",
}

// IsReadOnlyCommand reports whether cmd starts with one of the
// documented
// read-only prefixes (case-insensitive on the first token).
func IsReadOnlyCommand(cmd string) bool {
    return startsWithAny(strings.TrimSpace(cmd),
        readOnlyCommands)
}

// IsWriteCommand reports whether cmd starts with one of the
// documented
// write prefixes.
func IsWriteCommand(cmd string) bool {
    return startsWithAny(strings.TrimSpace(cmd), writeCommands)
}

func startsWithAny(cmd string, prefixes []string) bool {
    lower := strings.ToLower(cmd)
    for _, p := range prefixes {
        pl := strings.ToLower(p)
        if lower == pl || strings.HasPrefix(lower, pl+" ") ||
            strings.HasPrefix(lower, pl+"\t") {

```

```

        return true
    }
}
return false
}

// PresetRules returns the built-in rule slice for a named
// preset.
// Returns nil for unknown names.
func PresetRules(mode string) []Rule {
    switch mode {
    case "default":
        return nil
    case "auto":
        return []Rule{
            {Pattern: "*(*)", Action: confirmation.ActionAllow,
            Priority: 0,
            Description: "auto-allow everything (auto
            preset)", Source: ScopePreset},
        }
    case "acceptEdits":
        rules := []Rule{
            {Pattern: "Edit(*)", Action:
            confirmation.ActionAllow, Priority: 100,
            Description: "auto-allow Edit (acceptEdits
            preset)", Source: ScopePreset},
            {Pattern: "Write(*)", Action:
            confirmation.ActionAllow, Priority: 100,
            Description: "auto-allow Write (acceptEdits
            preset)", Source: ScopePreset},
            {Pattern: "MultiEdit(*)", Action:
            confirmation.ActionAllow, Priority: 100,
            Description: "auto-allow MultiEdit (acceptEdits
            preset)", Source: ScopePreset},
        }
        for _, p := range writeCommands {
            rules = append(rules, Rule{
                Pattern: "Bash(" + p + "*)",
                Action: confirmation.ActionAllow,
                Priority: 50,
                Description: "auto-allow write Bash (" + p + "*)
                (acceptEdits preset)",
                Source: ScopePreset,
            })
        }
        return rules
    case "dontAsk":
        rules := []Rule{

```

```

        {Pattern: "Read(*)", Action:
confirmation.ActionAllow, Priority: 100,
        Description: "auto-allow Read (dontAsk preset)",
Source: ScopePreset},
        {Pattern: "Glob(*)", Action:
confirmation.ActionAllow, Priority: 100,
        Description: "auto-allow Glob (dontAsk preset)",
Source: ScopePreset},
        {Pattern: "Grep(*)", Action:
confirmation.ActionAllow, Priority: 100,
        Description: "auto-allow Grep (dontAsk preset)",
Source: ScopePreset},
    }
    for _, p := range readOnlyCommands {
        rules = append(rules, Rule{
            Pattern: "Bash(" + p + "*)",
            Action: confirmation.ActionAllow,
            Priority: 50,
            Description: "auto-allow read-only Bash (" + p +
                "*) (dontAsk preset)",
            Source: ScopePreset,
        })
    }
    return rules
case "bypassPermissions":
    return []Rule{
        {Pattern: "*(*)", Action: confirmation.ActionAllow,
Priority: 1_000_000,
            Description: "BYPASS – auto-allow everything
(operator-only safety hatch)",
            Source: ScopePreset},
    }
}
return nil
}

```



Step 4: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -race -run 'TestPresetRules|
TestReadOnlyCommands|TestWriteCommands' ./internal/tools/
permissions/

```

Expected: PASS (8 tests).



Step 5: Run the whole permissions package

```
cd HelixCode && go test -count=1 -race ./internal/tools/permissions/...
```

Expected: PASS.



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" internal/tools/permissions/ && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 7: Commit

```
git -C HelixCode add internal/tools/permissions/mode_presets.go
internal/tools/permissions/mode_presets_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T06): five mode presets + conservative command lists

PresetRules returns the built-in rule slice for default | auto |
acceptEdits | dontAsk | bypassPermissions. acceptEdits and
dontAsk
expand the conservative readOnlyCommands / writeCommands lists
into
Bash(<prefix>*) allow rules. bypassPermissions uses priority
1_000_000
so it cannot be accidentally beaten by a user rule. 8 unit tests.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
EOF
)"
```

Task 7: Rule loader — YAML + file precedence (TDD)

Files: - Create: helix_code/internal/tools/permissions/rule_loader.go -
Test: helix_code/internal/tools/permissions/rule_loader_test.go



Step 1: Write failing test

Create helix_code/internal/tools/permissions/rule_loader_test.go:

```
package permissions

import (
```

```

"context"
"os"
"path/filepath"
"testing"

"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"

"dev.helix.code/internal/tools/confirmation"
)

func TestLoad_BothFilesMissingUsesPresetOnly(t *testing.T) {
    tmp := t.TempDir()
    loader := &FileLoader{
        UserPath:    filepath.Join(tmp, "user.yaml"),
        ProjectPath: filepath.Join(tmp, "project.yaml"),
        Mode:        "default",
    }
    rs, err := loader.Load(context.Background())
    require.NoError(t, err)
    assert.Equal(t, "default", rs.Mode)
    assert.Empty(t, rs.Rules, "default preset has no rules")
    assert.Empty(t, rs.Sources)
}

func TestLoad_UserFileOnly(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
mode: default
rules:
- pattern: "Bash(git status*)"
  action: allow
  priority: 100
  description: "user-level git status"
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing"), Mode: ""}
    rs, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, rs.Rules, 1)
    assert.Equal(t, "Bash(git status*)", rs.Rules[0].Pattern)
    assert.Equal(t, confirmation.ActionAllow, rs.Rules[0].Action)
    assert.Equal(t, ScopeUser, rs.Rules[0].Source)
}

func TestLoad_ProjectOverridesUserSamePattern(t *testing.T) {

```



```

    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    projPath := filepath.Join(tmp, "project.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
rules:
  - pattern: "Bash(rm*)"
    action: allow
`), 0o600))
    require.NoError(t, os.WriteFile(projPath,
        []byte(`apiVersion: helixcode.permissions/v1
rules:
  - pattern: "Bash(rm*)"
    action: deny
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        projPath}
    rs, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, rs.Rules, 1, "project replaces user for
        identical pattern")
    assert.Equal(t, confirmation.ActionDeny, rs.Rules[0].Action)
    assert.Equal(t, ScopeProject, rs.Rules[0].Source)
}

func TestLoad_MalformedYAMLError(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte("not: valid: yaml: ["), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing")}
    _, err := loader.Load(context.Background())
    assert.Error(t, err)
}

func TestLoad_UnknownPresetIsError(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
mode: nonsense
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing")}
    _, err := loader.Load(context.Background())
    assert.Error(t, err)
}

```

```

func TestLoad_UnknownAPIVersionIsError(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v999
mode: default
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing")}
    _, err := loader.Load(context.Background())
    assert.Error(t, err)
}

func TestLoad_PresetRulesIncluded(t *testing.T) {
    tmp := t.TempDir()
    loader := &FileLoader{
        UserPath:      filepath.Join(tmp, "user.yaml"),
        ProjectPath:    filepath.Join(tmp, "project.yaml"),
        Mode:           "dontAsk",
    }
    rs, err := loader.Load(context.Background())
    require.NoError(t, err)
    assert.Equal(t, "dontAsk", rs.Mode)
    patterns := patternsOf(rs.Rules)
    assert.Contains(t, patterns, "Read(*)")
    assert.Contains(t, patterns, "Glob(*)")
}

func TestSave_WritesFileWith0600Mode(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "perms", "user.yaml")
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing")}
    rule := Rule{
        Pattern:      "Bash(git status*)",
        Action:       confirmation.ActionAllow,
        Priority:     100,
        Description:  "saved by test",
    }
    require.NoError(t, loader.Save(context.Background(),
        ScopeUser, rule))
    info, err := os.Stat(userPath)
    require.NoError(t, err)
    assert.Equal(t, os.FileMode(0o600), info.Mode().Perm())
    dirInfo, err := os.Stat(filepath.Dir(userPath))
    require.NoError(t, err)
}

```

```
    assert.Equal(t, os.FileMode(0o700), dirInfo.Mode().Perm())
}
```



Step 2: Run test to verify it fails

```
cd HelixCode && go test -count=1 -run 'TestLoad|TestSave' ./
    internal/tools/permissions/
```

Expected: FAIL — FileLoader undefined.



Step 3: Implement rule_loader.go

Create helix_code/internal/tools/permissions/rule_loader.go:

```
package permissions

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"

    "gopkg.in/yaml.v3"

    "dev.helix.code/internal/tools/confirmation"
)

const expectedAPIVersion = "helixcode.permissions/v1"

// fileSchema is the on-disk YAML structure for a permissions
// file.
type fileSchema struct {
    APIVersion string `yaml:"apiVersion"`
    Mode       string `yaml:"mode"`
    Rules      []ruleSchema `yaml:"rules"`
}

type ruleSchema struct {
    Pattern      string `yaml:"pattern"`
    Action       string `yaml:"action"`
    Priority      int    `yaml:"priority"`
    Description  string `yaml:"description"`
}

// FileLoader reads layered permission files.
type FileLoader struct {
    UserPath string
}
```

```

ProjectPath string
Mode         string // overrides the file's mode: key when
                  non-empty (e.g. CLI flag)
}

// Load merges user + project files with the selected mode
// preset.
// Returns an error for malformed YAML, unknown apiVersion, or
// unknown preset.
// A missing file is treated as empty (not an error).
func (l *FileLoader) Load(ctx context.Context) (*RuleSet, error)
{
    userFile, err := readFileIfExists(l.UserPath)
    if err != nil {
        return nil, fmt.Errorf("reading user file %s: %w",
            l.UserPath, err)
    }
    projectFile, err := readFileIfExists(l.ProjectPath)
    if err != nil {
        return nil, fmt.Errorf("reading project file %s: %w",
            l.ProjectPath, err)
    }

    var sources []string
    if userFile != nil {
        if err := validateAPIVersion(userFile.APIVersion); err !=
            nil {
            return nil, fmt.Errorf("%s: %w", l.UserPath, err)
        }
        sources = append(sources, l.UserPath)
    }
    if projectFile != nil {
        if err := validateAPIVersion(projectFile.APIVersion);
            err != nil {
            return nil, fmt.Errorf("%s: %w", l.ProjectPath, err)
        }
        sources = append(sources, l.ProjectPath)
    }

    mode := l.Mode
    if mode == "" && projectFile != nil && projectFile.Mode !=
        "" {
        mode = projectFile.Mode
    }
    if mode == "" && userFile != nil && userFile.Mode != "" {
        mode = userFile.Mode
    }
    if mode == "" {

```

```

        mode = "default"
    }
    if !IsValidMode(mode) {
        return nil, fmt.Errorf("unknown permission mode %q
            (valid: %s)", mode, strings.Join(ValidModes, ", "))
    }

    merged, err := mergeRules(projectFile, userFile)
    if err != nil {
        return nil, err
    }
    merged = append(merged, PresetRules(mode)...)

    return &RuleSet{Mode: mode, Rules: merged, Sources:
        sources}, nil
}

// Save writes a single rule to the user or project file,
// creating it if
// missing. Directory mode 0700, file mode 0600.
func (l *FileLoader) Save(ctx context.Context, scope Scope, rule
    Rule) error {
    var path string
    switch scope {
    case ScopeUser:
        path = l.UserPath
    case ScopeProject:
        path = l.ProjectPath
    default:
        return fmt.Errorf("scope %s is not persistable (use user
            or project)", scope)
    }
    if path == "" {
        return fmt.Errorf("scope %s has no configured path",
            scope)
    }
    if err := os.MkdirAll(filepath.Dir(path), 0o700); err != nil
    {
        return fmt.Errorf("creating dir for %s: %w", path, err)
    }
    existing, err := readFileIfExists(path)
    if err != nil {
        return err
    }
    if existing == nil {
        existing = &fileSchema{APIVersion: expectedAPIVersion}
    }
    replaced := false

```

```

    for i, r := range existing.Rules {
        if r.Pattern == rule.Pattern {
            existing.Rules[i] = ruleToSchema(rule)
            replaced = true
            break
        }
    }
    if !replaced {
        existing.Rules = append(existing.Rules,
            ruleToSchema(rule))
    }
    out, err := yaml.Marshal(existing)
    if err != nil {
        return fmt.Errorf("marshalling %s: %w", path, err)
    }
    if err := os.WriteFile(path, out, 0o600); err != nil {
        return fmt.Errorf("writing %s: %w", path, err)
    }
    return nil
}

// Remove deletes the rule with the given pattern from the chosen
// scope's file.
// No-op (no error) if the rule does not exist.
func (l *FileLoader) Remove(ctx context.Context, scope Scope,
    pattern string) error {
    var path string
    switch scope {
    case ScopeUser:
        path = l.UserPath
    case ScopeProject:
        path = l.ProjectPath
    default:
        return fmt.Errorf("scope %s is not persistable", scope)
    }
    existing, err := readFileIfExists(path)
    if err != nil || existing == nil {
        return err
    }
    out := existing.Rules[:0]
    for _, r := range existing.Rules {
        if r.Pattern != pattern {
            out = append(out, r)
        }
    }
    existing.Rules = out
    body, err := yaml.Marshal(existing)
    if err != nil {

```

```

        return err
    }
    return os.WriteFile(path, body, 0o600)
}

func readFileIfExists(path string) (*fileSchema, error) {
    if path == "" {
        return nil, nil
    }
    body, err := os.ReadFile(path)
    if err != nil {
        if os.IsNotExist(err) {
            return nil, nil
        }
        return nil, err
    }
    var f fileSchema
    if err := yaml.Unmarshal(body, &f); err != nil {
        return nil, fmt.Errorf("yaml: %w", err)
    }
    return &f, nil
}

func validateAPIVersion(v string) error {
    if v == "" {
        return fmt.Errorf("missing apiVersion (expected %q)",
            expectedAPIVersion)
    }
    if v != expectedAPIVersion {
        return fmt.Errorf("unsupported apiVersion %q (expected %q)", v, expectedAPIVersion)
    }
    return nil
}

func mergeRules(project, user *fileSchema) ([]Rule, error) {
    var merged []Rule
    projectPatterns := map[string]bool{}
    if project != nil {
        for _, r := range project.Rules {
            converted, err := schemaToRule(r, ScopeProject)
            if err != nil {
                return nil, err
            }
            merged = append(merged, converted)
            projectPatterns[r.Pattern] = true
        }
    }
}

```

```

    if user != nil {
        for _, r := range user.Rules {
            if projectPatterns[r.Pattern] {
                continue
            }
            converted, err := schemaToRule(r, ScopeUser)
            if err != nil {
                return nil, err
            }
            merged = append(merged, converted)
        }
    }
    return merged, nil
}

func schemaToRule(s ruleSchema, source Scope) (Rule, error) {
    action, err := parseAction(s.Action)
    if err != nil {
        return Rule{}, fmt.Errorf("rule %q: %w", s.Pattern, err)
    }
    if _, err := ParsePattern(s.Pattern); err != nil {
        return Rule{}, err
    }
    return Rule{
        Pattern:      s.Pattern,
        Action:       action,
        Priority:     s.Priority,
        Description:  s.Description,
        Source:      source,
    }, nil
}

func ruleToSchema(r Rule) ruleSchema {
    return ruleSchema{
        Pattern:      r.Pattern,
        Action:       actionString(r.Action),
        Priority:     r.Priority,
        Description:  r.Description,
    }
}

func parseAction(s string) (confirmation.Action, error) {
    switch strings.ToLower(strings.TrimSpace(s)) {
    case "allow":
        return confirmation.ActionAllow, nil
    case "ask":
        return confirmation.ActionAsk, nil
    case "deny":

```



```

        return confirmation.ActionDeny, nil
    }
    return 0, fmt.Errorf("invalid action %q (expected allow|ask|deny)", s)
}

func actionString(a confirmation.Action) string {
    switch a {
    case confirmation.ActionAllow:
        return "allow"
    case confirmation.ActionAsk:
        return "ask"
    case confirmation.ActionDeny:
        return "deny"
    }
    return "ask"
}

```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -race -run 'TestLoad|TestSave' ./internal/tools/permissions/
```

Expected: PASS (8 tests).



Step 5: Run the whole package

```
cd HelixCode && go test -count=1 -race ./internal/tools/permissions/...
```

Expected: PASS.



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" internal/tools/permissions/ && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 7: Commit

```
git -C HelixCode add internal/tools/permissions/rule_loader.go
internal/tools/permissions/rule_loader_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T07): YAML rule loader with project-over-user
precedence"
```

```
FileLoader.Load reads user + project YAML, validates apiVersion
helixcode.permissions/v1, merges rules (project overrides user on
identical pattern), then appends mode preset rules. Save writes a
single rule with file mode 0600 in a 0700 directory. 8 unit tests
covering missing files, malformed YAML, unknown apiVersion /
    preset,
identical-pattern override, and Save permission modes.
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 8: Permissions facade — register Policy with PolicyEngine

Files: - Create: helix_code/internal/tools/permissions/permissions.go -
Test: helix_code/internal/tools/permissions/permissions_test.go



Step 1: Write failing test

Create helix_code/internal/tools/permissions/permissions_test.go:

```
package permissions

import (
    "context"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/tools/confirmation"
)

func TestNewEngine_RegistersPolicy(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
mode: default
rules:
  - pattern: "Bash(git status*)"
    action: allow
```

```

        priority: 100
    `), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing")}

    policyEngine := confirmation.NewPolicyEngine()
    eng, err := NewEngine(context.Background(), loader,
        policyEngine)
    require.NoError(t, err)
    require.NotNil(t, eng)

    req := confirmation.ConfirmationRequest{
        ToolName:    "Bash",
        Parameters: map[string]interface{}{"command":
            "git status -sb"},
    }
    decision, err := policyEngine.Evaluate(req)
    require.NoError(t, err)
    assert.Equal(t, confirmation.ActionAllow, decision.Action)
}

func TestNewEngine_EvaluatesViaRuleEngine(t *testing.T) {
    tmp := t.TempDir()
    loader := &FileLoader{
        UserPath:    filepath.Join(tmp, "user.yaml"),
        ProjectPath: filepath.Join(tmp, "project.yaml"),
        Mode:        "dontAsk",
    }
    pe := confirmation.NewPolicyEngine()
    _, err := NewEngine(context.Background(), loader, pe)
    require.NoError(t, err)

    req := confirmation.ConfirmationRequest{
        ToolName:    "Read",
        Parameters: map[string]interface{}{"path": "/etc/hosts"},
    }
    decision, err := pe.Evaluate(req)
    require.NoError(t, err)
    assert.Equal(t, confirmation.ActionAllow, decision.Action)
}

func TestNewEngine_DenyRuleBlocksMatch(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
mode: auto
rules:

```

```

- pattern: "Bash(rm*)"
  action: deny
  priority: 1000
`), 0o600))
  loader := &FileLoader{UserPath: userPath, ProjectPath:
    filepath.Join(tmp, "missing")}
  pe := confirmation.NewPolicyEngine()
  _, err := NewEngine(context.Background(), loader, pe)
  require.NoError(t, err)

  req := confirmation.ConfirmationRequest{
    ToolName: "Bash",
    Parameters: map[string]interface{}{"command": "rm -rf /
      tmp/x"},
  }
  decision, err := pe.Evaluate(req)
  require.NoError(t, err)
  assert.Equal(t, confirmation.ActionDeny, decision.Action)
}

```



Step 2: Run test to verify it fails

```
cd HelixCode && go test -count=1 -run 'TestNewEngine' ./internal/
tools/permissions/
```

Expected: FAIL — NewEngine undefined.



Step 3: Implement permissions.go

Create helix_code/internal/tools/permissions/permissions.go:

```

package permissions

import (
    "context"
    "fmt"

    "dev.helix.code/internal/tools/confirmation"
)

// Engine is the public facade. Construct with NewEngine; it
// registers a
// confirmation.Policy with the supplied PolicyEngine and is
// ready to use.
type Engine struct {
    ruleEngine    *RuleEngine
    policyEngine  *confirmation.PolicyEngine
    loader        *FileLoader
}

```

```

    ruleSet      *RuleSet
}

// PolicyName is the registered policy name in the
// confirmation.PolicyEngine.
const PolicyName = "permissions/rule-engine"

// NewEngine loads rules via the loader, builds a RuleEngine, and
// registers a
// confirmation.Policy with the supplied PolicyEngine.
func NewEngine(ctx context.Context, loader *FileLoader, pe
    *confirmation.PolicyEngine) (*Engine, error) {
    rs, err := loader.Load(ctx)
    if err != nil {
        return nil, fmt.Errorf("loading permission rules: %w",
            err)
    }
    re, err := NewRuleEngine(rs.Rules)
    if err != nil {
        return nil, fmt.Errorf("building rule engine: %w", err)
    }
    eng := &Engine{
        ruleEngine:  re,
        policyEngine: pe,
        loader:      loader,
        ruleSet:     rs,
    }
    policy := eng.buildPolicy()
    pe.SetPolicy("*", policy) // applies to every tool
    return eng, nil
}

// RuleSet returns the loaded RuleSet (for inspection by
// `permissions list`).
func (e *Engine) RuleSet() *RuleSet { return e.ruleSet }

// buildPolicy converts the rule engine into a
// confirmation.Policy with one
// custom-condition rule that delegates the decision to
// e.ruleEngine.
func (e *Engine) buildPolicy() *confirmation.Policy {
    return &confirmation.Policy{
        Name:          PolicyName,
        Description:    "claude-code-style permission rules
            (mode " + e.ruleSet.Mode + ")",
        Enabled:       true,
        DefaultAction: confirmation.ActionAsk,
        BatchDefaultAction: confirmation.ActionDeny,
    }
}

```

```

Rules: []confirmation.Rule{
    {
        Name:      PolicyName + "/dispatch",
        Priority: 1_000_000_000, // beats every built-in
confirmation rule
        Condition: confirmation.Condition{
            Custom: func(req
confirmation.ConfirmationRequest) bool { return true },
            },
        Action: confirmation.ActionAsk, // overridden by
Custom below
            },
        },
    },
}
}

```

The above produces a single dispatch rule whose only purpose is to *match*. We need the `confirmation.PolicyEngine.Evaluate` to actually call our `RuleEngine`. The cleanest way is to expose the dispatch through `Custom` predicate side-effect — but `Condition.Matches` returning true forces the rule's static `Action`, not a dynamic one.

Use the existing `Custom` predicate as the *match gate* and put the real decision into a tiny lookup the policy engine consults at evaluate-time. Add this **before** `buildPolicy`:

```

// Decide is the public single-shot decision API used by the
// dispatch rule's
// custom condition; it is also what the slash command and CLI
// subcommands call.
func (e *Engine) Decide(req confirmation.ConfirmationRequest)
    Decision {
    primary := primaryParam(req)
    return e.ruleEngine.Evaluate(req.ToolName, primary)
}

func primaryParam(req confirmation.ConfirmationRequest) string {
    if cmd, ok := req.Parameters["command"].(string); ok {
        return cmd
    }
    if p, ok := req.Parameters["path"].(string); ok {
        return p
    }
    if p, ok := req.Parameters["file_path"].(string); ok {
        return p
    }
    return req.Operation.Target
}

```

Replace `buildPolicy` with one that uses three rules — one per `Action` — and routes via `Custom`:

```

func (e *Engine) buildPolicy() *confirmation.Policy {
    deny := func(req confirmation.ConfirmationRequest) bool {
        return e.Decide(req).Action == confirmation.ActionDeny
    }
    allow := func(req confirmation.ConfirmationRequest) bool {
        return e.Decide(req).Action == confirmation.ActionAllow
    }
    ask := func(req confirmation.ConfirmationRequest) bool {
        return e.Decide(req).Action == confirmation.ActionAsk
    }
    return &confirmation.Policy{
        Name:          PolicyName,
        Description:    "claude-code-style permission rules\n(mode " + e.ruleSet.Mode + ")",
        Enabled:        true,
        DefaultAction:  confirmation.ActionAsk,
        BatchDefaultAction: confirmation.ActionDeny,
        Rules: []confirmation.Rule{
            {
                Name:      PolicyName + "/deny",
                Priority:  1_000_000_002,
                Condition: confirmation.Condition{Custom: deny},
                Action:    confirmation.ActionDeny,
            },
            {
                Name:      PolicyName + "/allow",
                Priority: 1_000_000_001,
                Condition: confirmation.Condition{Custom: allow},
                Action:    confirmation.ActionAllow,
            },
            {
                Name:      PolicyName + "/ask",
                Priority: 1_000_000_000,
                Condition: confirmation.Condition{Custom: ask},
                Action:    confirmation.ActionAsk,
            },
        },
    }
}

```

The deny rule has highest priority so a shell-parse error or smuggle-deny short-circuits before allow. Each rule's Custom predicate calls Decide once; the existing PolicyEngine already iterates rules in priority order and returns on the first match.



Step 4: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -race -run 'TestNewEngine' ./
internal/tools/permissions/

```

Expected: PASS (3 tests).



Step 5: Run the whole package + confirmation regression

```
cd HelixCode && go test -count=1 -race ./internal/tools/permissions/... ./internal/tools/confirmation/...
```

Expected: PASS everywhere.



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" internal/tools/permissions/ && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 7: Commit

```
git -C HelixCode add internal/tools/permissions/permissions.go
internal/tools/permissions/permissions_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T08): permissions.Engine facade registers Policy with
PolicyEngine
```

```
NewEngine loads rules, builds a RuleEngine, and registers a
three-rule
confirmation.Policy (deny > allow > ask, each a Custom-predicate
that
delegates to RuleEngine.Evaluate). Decide() is the single-shot
decision
API used by the dispatch rule, and is exposed for the upcoming
CLI
subcommands and slash command. 3 integration-style unit tests
covering
allow rule, preset-only dontAsk, and deny-overrides-auto.
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
```

```
EOF
)"
```

Task 9: Wire –permission-mode flag + integration test (no mocks)

Files: - Modify: `helix_code/cmd/cli/main.go` (add flag + `permissions.Engine` bootstrap) - Test: `helix_code/tests/integration/permissions/permissions_integration_test.go`



Step 1: Locate the CLI flag block

```
cd HelixCode && grep -n 'pflag\|StringVar\|cobra' cmd/cli/main.go | head -20
```

Note the line range where flags are declared and the function that builds the CLI value.



Step 2: Add the –permission-mode flag

In `helix_code/cmd/cli/main.go`, find the flag-registration block (where existing flags are added with `flags.StringVar` or `cmd.Flags().StringVar`). Add:

```
// Permission mode preset (default | auto | acceptEdits | dontAsk | bypassPermissions)
flags.StringVar(&cli.permissionMode, "permission-mode", "",
    "permission preset: default|auto|acceptEdits|dontAsk|bypassPermissions")
```

(Adjust the receiver name to match the surrounding code.)

Add the field to the CLI struct (search for the struct that holds other flag values):

```
permissionMode string
```



Step 3: Bootstrap the `permissions.Engine` on startup

In the CLI's startup function (where other engines/managers are constructed), add:

```
import (
    // existing imports
    "dev.helix.code/internal/tools/permissions"
    "dev.helix.code/internal/tools/confirmation"
    // ...
)

func (c *CLI) initPermissions(ctx context.Context, policyEngine *confirmation.PolicyEngine) error {
    if c.permissionMode != "" && !
        permissions.IsValidMode(c.permissionMode) {
        return fmt.Errorf("invalid --permission-mode %q (valid: %v)",
            c.permissionMode, permissions.ValidModes())
    }
}
```

```

        c.permissionMode, permissions.ValidModes)
    }
    home, err := os.UserHomeDir()
    if err != nil {
        return fmt.Errorf("resolving user home dir: %w", err)
    }
    cwd, err := os.Getwd()
    if err != nil {
        return fmt.Errorf("resolving cwd: %w", err)
    }
    loader := &permissions.FileLoader{
        UserPath:    filepath.Join(home, ".helixcode",
            "permissions.yaml"),
        ProjectPath: filepath.Join(cwd, ".helixcode",
            "permissions.yaml"),
        Mode:        c.permissionMode,
    }
    eng, err := permissions.NewEngine(ctx, loader, policyEngine)
    if err != nil {
        return fmt.Errorf("initialising permissions: %w", err)
    }
    c.permissionsEngine = eng
    return nil
}

```

Add the `permissionsEngine *permissions.Engine` field to the CLI struct, and call `initPermissions` from the existing CLI startup sequence (after the policy engine is created, before tool execution begins).



Step 4: Verify it compiles

```
cd HelixCode && go build ./cmd/cli/...
```

Expected: clean compile.



Step 5: Write integration test

Create directory:

```
mkdir -p helix_code/tests/integration/permissions
```

Create `helix_code/tests/integration/permissions/permissions_integration_test.go`:

```
//go:build integration
```

```
package permissions_test
```

```
import (
```

```

"context"
"os"
"os/exec"
"path/filepath"
"testing"

"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"

"dev.helix.code/internal/tools/confirmation"
"dev.helix.code/internal/tools/permissions"
)

// TestIntegration_DenyRuleBlocksRealOSExec proves that a deny
// rule prevents
// a real os/exec invocation against a real temp filesystem. NO
// mocks.
func TestIntegration_DenyRuleBlocksRealOSExec(t *testing.T) {
    tmp := t.TempDir()
    marker := filepath.Join(tmp, "marker")
    require.NoError(t, os.WriteFile(marker, []byte("present"),
        0o644))

    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
mode: auto
rules:
- pattern: "Bash(rm*)"
  action: deny
  priority: 1000
`), 0o600))

    loader := &permissions.FileLoader{UserPath: userPath,
        ProjectPath: filepath.Join(tmp, "missing")}
    pe := confirmation.NewPolicyEngine()
    _, err := permissions.NewEngine(context.Background(),
        loader, pe)
    require.NoError(t, err)

    req := confirmation.ConfirmationRequest{
        ToolName: "Bash",
        Parameters: map[string]interface{}{"command": "rm -rf "
            + marker},
    }
    decision, err := pe.Evaluate(req)
    require.NoError(t, err)
    require.Equal(t, confirmation.ActionDeny, decision.Action)
}

```

```

// PROOF: the marker is still on disk because the engine
// denied the call.
// We deliberately do NOT call exec here – that would be the
// bluff.
// Instead, we assert the engine prevents us from reaching
// exec.
_, statErr := os.Stat(marker)
assert.NoError(t, statErr, "marker must still exist; deny
    rule blocked exec")

// Belt-and-braces: if a caller were to ignore the decision
// and shell out,
// the file WOULD be deleted. Verify the test scenario itself
// is realistic
// by running the inverse: an ALLOWED command actually
// changes the FS.
allowMarker := filepath.Join(tmp, "allow-marker")
require.NoError(t, os.WriteFile(allowMarker,
    []byte("present"), 0o644))
out, err := exec.Command("rm", "-f",
    allowMarker).CombinedOutput()
require.NoError(t, err, "control: real exec works: %s", out)
_, statErr2 := os.Stat(allowMarker)
assert.True(t, os.IsNotExist(statErr2),
    "control: rm actually deletes")
}

// TestIntegration_SmuggleViaCommandSubstitutionDenied ensures
// the engine
// cannot be tricked by $() substitution.
func TestIntegration_SmuggleViaCommandSubstitutionDenied(t
    *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
mode: auto
rules:
  - pattern: "Bash(rm*)"
    action: deny
    priority: 1000
`), 0o600))
    loader := &permissions.FileLoader{UserPath: userPath,
        ProjectPath: filepath.Join(tmp, "missing")}
    pe := confirmation.NewPolicyEngine()
    _, err := permissions.NewEngine(context.Background(),
        loader, pe)

```

```

require.NoError(t, err)

req := confirmation.ConfirmationRequest{
    ToolName:    "Bash",
    Parameters: map[string]interface{}{"command": "echo hi $(rm -rf /tmp/smuggled)"},
}
decision, err := pe.Evaluate(req)
require.NoError(t, err)
assert.Equal(t, confirmation.ActionDeny, decision.Action,
    "smuggled rm in command substitution must propagate deny")
}

// TestIntegration_ReadOnlyDontAskAllowsLs confirms the dontAsk
// preset
// auto-allows real read-only commands.
func TestIntegration_ReadOnlyDontAskAllowsLs(t *testing.T) {
    tmp := t.TempDir()
    loader := &permissions.FileLoader{
        UserPath:    filepath.Join(tmp, "user.yaml"),
        ProjectPath: filepath.Join(tmp, "project.yaml"),
        Mode:        "dontAsk",
    }
    pe := confirmation.NewPolicyEngine()
    _, err := permissions.NewEngine(context.Background(),
        loader, pe)
    require.NoError(t, err)

    req := confirmation.ConfirmationRequest{
        ToolName:    "Bash",
        Parameters: map[string]interface{}{"command": "ls
        -la /"},
    }
    decision, err := pe.Evaluate(req)
    require.NoError(t, err)
    assert.Equal(t, confirmation.ActionAllow, decision.Action)
}

```



Step 6: Run integration tests

```

cd HelixCode && go test -count=1 -race -v -tags=integration ./
    tests/integration/permissions/...

```

Expected: PASS (3 tests). NO mocks anywhere.



Step 7: Verify the integration tests really fail when the engine is broken (CONST-035 mutation check)

Temporarily change the deny rule's priority in `mode_presets.go` (or whichever just-edited file) to be lower than auto's allow, then re-run:

```
cd HelixCode && go test -count=1 -race -v -tags=integration ./
tests/integration/permissions/...
```

Expected: FAIL — the smuggle test should now pass Allow. **Revert** the mutation. Re-run; expect PASS again.

This is documentation-only — DON'T commit the mutation. Only the unmutated code goes in the commit.



Step 8: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" cmd/cli/main.go internal/tools/permissions/
tests/integration/permissions/ && echo "BLUFF FOUND" ||
echo "clean"
```

Expected: clean.



Step 9: Commit

```
git -C HelixCode add cmd/cli/main.go tests/integration/
permissions/
git commit -m "$(cat <<'EOF'
feat(P1-F02-T09): wire --permission-mode flag + integration tests
(no mocks)
```

```
CLI bootstrap loads ~/.helixcode/permissions.yaml + project file,
then
registers permissions.Engine with the confirmation.PolicyEngine
before
tool execution. Three integration tests with -tags=integration
and NO
mocks: deny blocks real exec, command-substitution smuggle is
denied,
dontAsk preset auto-allows ls. Mutation-tested locally (smuggle
test
fails when rule priority is broken; reverted).
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
```

```
EOF
)"
```

Task 10: helixcode permissions {list,add,remove,check} subcommands

Files: - Create: helix_code/cmd/cli/permissions_cmd.go - Test: helix_code/cmd/cli/permissions_cmd_test.go



Step 1: Write failing test for list

Create helix_code/cmd/cli/permissions_cmd_test.go:

```
package main

import (
    "bytes"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestPermissionsList_ShowsLoadedRules(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.permissions/v1
mode: default
rules:
  - pattern: "Bash(git status*)"
    action: allow
    priority: 100
    description: "user-level git status"
`), 0o600))

    var buf bytes.Buffer
    err := runPermissionsList(&buf, userPath, filepath.Join(tmp,
        "missing"), "")
    require.NoError(t, err)

    out := buf.String()
    assert.Contains(t, out, "Bash(git status*)")
    assert.Contains(t, out, "allow")
    assert.Contains(t, out, "user")
}

func TestPermissionsAdd_WritesToUserFile(t *testing.T) {
    tmp := t.TempDir()
```

```

    userPath := filepath.Join(tmp, "user.yaml")
    err := runPermissionsAdd(userPath, "Bash(git status*)",
        "allow", 100, "added by test")
    require.NoError(t, err)
    body, err := os.ReadFile(userPath)
    require.NoError(t, err)
    assert.Contains(t, string(body), "Bash(git status*)")
    assert.Contains(t, string(body), "allow")
}

func TestPermissionsRemove_DropsRule(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, runPermissionsAdd(userPath, "Bash(rm*)",
        "deny", 1000, ""))
    require.NoError(t, runPermissionsRemove(userPath,
        "Bash(rm*)"))
    body, err := os.ReadFile(userPath)
    require.NoError(t, err)
    assert.NotContains(t, string(body), "Bash(rm*)")
}

func TestPermissionsCheck_DryRun(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, runPermissionsAdd(userPath, "Bash(git
        status*)", "allow", 100, ""))

    var buf bytes.Buffer
    err := runPermissionsCheck(&buf, userPath,
        filepath.Join(tmp, "missing"), "", "Bash", "git status
        -sb")
    require.NoError(t, err)
    assert.Contains(t, buf.String(), "allow")
    assert.Contains(t, buf.String(), "Bash(git status*)")
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run 'TestPermissions' ./cmd/
cli/

```

Expected: FAIL — runPermissionsList etc. undefined.



Step 3: Implement permissions_cmd.go

Create helix_code/cmd/cli/permissions_cmd.go:


```

package main

import (
    "context"
    "fmt"
    "io"
    "os"
    "path/filepath"
    "text/tabwriter"

    "github.com/spf13/cobra"

    "dev.helix.code/internal/tools/confirmation"
    "dev.helix.code/internal/tools/permissions"
)

func newPermissionsCommand() *cobra.Command {
    cmd := &cobra.Command{
        Use:     "permissions",
        Short:   "Manage HelixCode permission rules",
    }
    cmd.AddCommand(newPermissionsListCommand())
    cmd.AddCommand(newPermissionsAddCommand())
    cmd.AddCommand(newPermissionsRemoveCommand())
    cmd.AddCommand(newPermissionsCheckCommand())
    return cmd
}

func newPermissionsListCommand() *cobra.Command {
    var mode string
    cmd := &cobra.Command{
        Use:     "list",
        Short:   "Show effective permission rules",
        RunE: func(cmd *cobra.Command, args []string) error {
            user, project := defaultPaths()
            return runPermissionsList(os.Stdout, user, project,
                mode)
        },
    }
    cmd.Flags().StringVar(&mode, "permission-mode", "",
        "override mode preset")
    return cmd
}

func newPermissionsAddCommand() *cobra.Command {
    var (
        scope    string
        priority int
    )

```

```

        descr    string
    )
    cmd := &cobra.Command{
        Use:    "add <pattern> <action>",
        Short:  "Add a permission rule",
        Args:   cobra.ExactArgs(2),
        RunE:   func(cmd *cobra.Command, args []string) error {
            path, err := pathForScope(scope)
            if err != nil {
                return err
            }
            return runPermissionsAdd(path, args[0], args[1],
                priority, descr)
        },
    }
    cmd.Flags().StringVar(&scope, "scope", "user", "user|
        project")
    cmd.Flags().IntVar(&priority, "priority", 0, "rule priority
        (higher wins)")
    cmd.Flags().StringVar(&descr, "description", "", "free-text
        description")
    return cmd
}

func newPermissionsRemoveCommand() *cobra.Command {
    var scope string
    cmd := &cobra.Command{
        Use:    "remove <pattern>",
        Short:  "Remove a permission rule",
        Args:   cobra.ExactArgs(1),
        RunE:   func(cmd *cobra.Command, args []string) error {
            path, err := pathForScope(scope)
            if err != nil {
                return err
            }
            return runPermissionsRemove(path, args[0])
        },
    }
    cmd.Flags().StringVar(&scope, "scope", "user", "user|
        project")
    return cmd
}

func newPermissionsCheckCommand() *cobra.Command {
    var (
        mode string
        body string
    )

```

```

cmd := &cobra.Command{
    Use:     "check <tool>",
    Short:   "Dry-run a tool call against the rules",
    Args:    cobra.ExactArgs(1),
    RunE:    func(cmd *cobra.Command, args []string) error {
        user, project := defaultPaths()
        return runPermissionsCheck(os.Stdout, user, project,
            mode, args[0], body)
    },
}

cmd.Flags().StringVar(&mode, "permission-mode", "",
    "override mode preset")
cmd.Flags().StringVar(&body, "command", "",
    "Bash command (or path for file tools)")
return cmd
}

func defaultPaths() (string, string) {
    home, _ := os.UserHomeDir()
    cwd, _ := os.Getwd()
    return filepath.Join(home, ".helixcode", "permissions.yaml"),
        filepath.Join(cwd, ".helixcode", "permissions.yaml")
}

func pathForScope(scope string) (string, error) {
    user, project := defaultPaths()
    switch scope {
    case "user":
        return user, nil
    case "project":
        return project, nil
    }
    return "", fmt.Errorf("unknown --scope %q (valid: user,
        project)", scope)
}

func runPermissionsList(out io.Writer, userPath, projectPath,
    mode string) error {
    loader := &permissions.FileLoader{UserPath: userPath,
        ProjectPath: projectPath, Mode: mode}
    rs, err := loader.Load(context.Background())
    if err != nil {
        return err
    }
    tw := tabwriter.NewWriter(out, 0, 0, 2, ' ', 0)
    fmt.Fprintf(tw,
        "PATTERN\tACTION\tPRIORITY\tSOURCE\tDESCRIPTION\n")
    for _, r := range rs.Rules {

```

```

        fmt.Fprintf(tw, "%s\t%s\t%d\t%s\t%s\n",
            r.Pattern, actionName(r.Action), r.Priority,
            r.Source, r.Description)
    }
    fmt.Fprintf(tw, "\nMode: %s\nSources: %v\n", rs.Mode,
        rs.Sources)
    return tw.Flush()
}

func runPermissionsAdd(path, pattern, action string, priority
    int, description string) error {
    if _, err := permissions.ParsePattern(pattern); err != nil {
        return err
    }
    a, err := actionFromString(action)
    if err != nil {
        return err
    }
    loader := &permissions.FileLoader{UserPath: path,
        ProjectPath: ""}
    if isProjectPath(path) {
        loader = &permissions.FileLoader{UserPath: "",
            ProjectPath: path}
    }
    scope := permissions.ScopeUser
    if isProjectPath(path) {
        scope = permissions.ScopeProject
    }
    return loader.Save(context.Background(), scope,
        permissions.Rule{
            Pattern:    pattern,
            Action:     a,
            Priority:   priority,
            Description: description,
        })
}

func runPermissionsRemove(path, pattern string) error {
    loader := &permissions.FileLoader{UserPath: path,
        ProjectPath: ""}
    scope := permissions.ScopeUser
    if isProjectPath(path) {
        loader = &permissions.FileLoader{UserPath: "",
            ProjectPath: path}
        scope = permissions.ScopeProject
    }
    return loader.Remove(context.Background(), scope, pattern)
}

```

```

func runPermissionsCheck(out io.Writer, userPath, projectPath,
    mode, tool, body string) error {
    loader := &permissions.FileLoader{UserPath: userPath,
        ProjectPath: projectPath, Mode: mode}
    pe := confirmation.NewPolicyEngine()
    eng, err := permissions.NewEngine(context.Background(),
        loader, pe)
    if err != nil {
        return err
    }
    req := confirmation.ConfirmationRequest{
        ToolName: tool,
        Parameters: map[string]interface{}{"command": body,
            "path": body, "file_path": body},
    }
    d := eng.Decide(req)
    fmt.Fprintf(out, "decision: %s\nmatched: %s\nreason: %s\n",
        actionName(d.Action), d.MatchedPattern, d.Reason)
    return nil
}

func isProjectPath(path string) bool {
    cwd, err := os.Getwd()
    if err != nil {
        return false
    }
    abs, err := filepath.Abs(path)
    if err != nil {
        return false
    }
    cwdAbs, _ := filepath.Abs(cwd)
    return filepath.HasPrefix(abs, cwdAbs)
}

func actionFromString(s string) (confirmation.Action, error) {
    switch s {
    case "allow":
        return confirmation.ActionAllow, nil
    case "ask":
        return confirmation.ActionAsk, nil
    case "deny":
        return confirmation.ActionDeny, nil
    }
    return 0, fmt.Errorf("invalid action %q (allow|ask|deny)", s)
}

func actionName(a confirmation.Action) string {

```

```

switch a {
case confirmation.ActionAllow:
    return "allow"
case confirmation.ActionAsk:
    return "ask"
case confirmation.ActionDeny:
    return "deny"
}
return "unknown"
}

```

In the CLI's root command setup (search for `rootCmd.AddCommand()`, add:

```
rootCmd.AddCommand(newPermissionsCommand())
```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -race -run 'TestPermissions' ./cmd/cli/
```

Expected: PASS (4 tests).



Step 5: Smoke-test the CLI binary

```
cd HelixCode && go build -o bin/helixcode ./cmd/cli && ./bin/helixcode permissions --help
```

Expected: subcommand list(list add remove check) printed.



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" cmd/cli/permissions_cmd.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 7: Commit

```
git -C HelixCode add cmd/cli/permissions_cmd.go cmd/cli/permissions_cmd_test.go cmd/cli/main.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T10): helixcode permissions {list,add,remove,check} subcommands
EOF")"
```

Cobra subcommand group writing/reading the same YAML files the runtime

loader uses. list shows effective rules + sources; add validates pattern
+ action before writing; remove drops by exact pattern; check is a
dry-run that prints decision + matched rule + reason. 4 unit tests
covering each subcommand. Smoke-tested via helixcode permissions
--help.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 11: /permissions slash command

Files: - Create: helix_code/internal/commands/permissions_command.go -
Test: helix_code/internal/commands/permissions_command_test.go



Step 1: Inspect the Command interface

```
cd HelixCode && head -80 internal/commands/command.go
```

Confirm the Command interface signature: Name, Aliases, Description, Usage, Execute(ctx, *CommandContext) (*CommandResult, error). Use this as the contract.



Step 2: Write failing test

Create helix_code/internal/commands/permissions_command_test.go:

```
package commands

import (
    "context"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestPermissionsCommand_Name(t *testing.T) {
    cmd := NewPermissionsCommand()
    assert.Equal(t, "permissions", cmd.Name())
}
```

```

func TestPermissionsCommand_ListSubaction(t *testing.T) {
    tmp := t.TempDir()
    t.Setenv("HOME", tmp)
    userDir := filepath.Join(tmp, ".helixcode")
    require.NoError(t, os.MkdirAll(userDir, 0o700))
    require.NoError(t, os.WriteFile(filepath.Join(userDir,
        "permissions.yaml"), []byte(`apiVersion:
        helixcode.permissions/v1
mode: default
rules:
  - pattern: "Bash(git status*)"
    action: allow
`), 0o600))

    cmd := NewPermissionsCommand()
    res, err := cmd.Execute(context.Background(),
        &CommandContext{
            Args: []string{},
            RawInput: "/permissions",
        })
    require.NoError(t, err)
    require.NotNil(t, res)
    assert.Contains(t, res.Output, "Bash(git status*)")
}

func TestPermissionsCommand_ModeSubaction(t *testing.T) {
    tmp := t.TempDir()
    t.Setenv("HOME", tmp)

    cmd := NewPermissionsCommand()
    res, err := cmd.Execute(context.Background(),
        &CommandContext{
            Args: []string{"mode", "dontAsk"},
            RawInput: "/permissions mode dontAsk",
        })
    require.NoError(t, err)
    assert.Contains(t, res.Output, "dontAsk")
}

func TestPermissionsCommand_RejectsUnknownMode(t *testing.T) {
    cmd := NewPermissionsCommand()
    res, err := cmd.Execute(context.Background(),
        &CommandContext{
            Args: []string{"mode", "nonsense"},
            RawInput: "/permissions mode nonsense",
        })
    assert.Error(t, err)
}

```



```

    if res != nil {
        assert.NotEmpty(t, res.Output)
    }
}

```



Step 3: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run 'TestPermissionsCommand' ./
internal/commands/

```

Expected: FAIL — NewPermissionsCommand undefined.



Step 4: Implement permissions_command.go

Create helix_code/internal/commands/permissions_command.go:

```

package commands

import (
    "bytes"
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "text/tabwriter"

    "dev.helix.code/internal/tools/confirmation"
    "dev.helix.code/internal/tools/permissions"
)

// PermissionsCommand implements /permissions.
//
// Subactions:
//   /permissions                - list effective rules
//   /permissions mode <preset>  - change session-only mode
//   /permissions add <pattern> <action> [priority] - add rule
//                                   (session-only)
//   /permissions remove <pattern> - remove rule (session-only)
//
// Persistent edits go through the `helixcode permissions` Cobra
// subcommand.
type PermissionsCommand struct {
    mode string
}

// NewPermissionsCommand constructs the /permissions slash
// command.

```

```

func NewPermissionsCommand() *PermissionsCommand {
    return &PermissionsCommand{}
}

func (c *PermissionsCommand) Name() string { return
    "permissions" }
func (c *PermissionsCommand) Aliases() []string { return
    []string{"perms"} }
func (c *PermissionsCommand) Description() string { return
    "manage permission rules" }
func (c *PermissionsCommand) Usage() string {
    return "/permissions [mode <preset> | add <pattern> <action>
    [priority] | remove <pattern>]"
}

func (c *PermissionsCommand) Execute(ctx context.Context, cmdCtx
    *CommandContext) (*CommandResult, error) {
    if len(cmdCtx.Args) == 0 {
        return c.list(ctx)
    }
    switch cmdCtx.Args[0] {
    case "mode":
        if len(cmdCtx.Args) < 2 {
            return nil, fmt.Errorf("usage: /permissions mode
            <preset>")
        }
        return c.setMode(cmdCtx.Args[1])
    case "add":
        if len(cmdCtx.Args) < 3 {
            return nil, fmt.Errorf("usage: /permissions add
            <pattern> <action> [priority]")
        }
        priority := 0
        if len(cmdCtx.Args) >= 4 {
            fmt.Sscanf(cmdCtx.Args[3], "%d", &priority)
        }
        return c.addSession(cmdCtx.Args[1], cmdCtx.Args[2],
            priority)
    case "remove":
        if len(cmdCtx.Args) < 2 {
            return nil, fmt.Errorf("usage: /permissions remove
            <pattern>")
        }
        return c.removeSession(cmdCtx.Args[1])
    default:
        return nil, fmt.Errorf("unknown subcommand %q (valid:
        mode, add, remove)", cmdCtx.Args[0])
    }
}

```

```

}

func (c *PermissionsCommand) list(ctx context.Context)
    (*CommandResult, error) {
    loader, err := defaultLoader(c.mode)
    if err != nil {
        return nil, err
    }
    rs, err := loader.Load(ctx)
    if err != nil {
        return nil, err
    }
    var buf bytes.Buffer
    tw := tabwriter.NewWriter(&buf, 0, 0, 2, ' ', 0)
    fmt.Fprintf(tw,
        "PATTERN\tACTION\tPRIORITY\tSOURCE\tDESCRIPTION\n")
    for _, r := range rs.Rules {
        fmt.Fprintf(tw, "%s\t%s\t%d\t%s\t%s\n",
            r.Pattern, actionToName(r.Action), r.Priority,
            r.Source, r.Description)
    }
    fmt.Fprintf(tw, "\nMode: %s\nSources: %s\n", rs.Mode,
        strings.Join(rs.Sources, ", "))
    tw.Flush()
    return &CommandResult{Output: buf.String()}, nil
}

func (c *PermissionsCommand) setMode(mode string)
    (*CommandResult, error) {
    if !permissions.IsValidMode(mode) {
        return nil, fmt.Errorf("unknown mode %q (valid: %v)",
            mode, permissions.ValidModes)
    }
    c.mode = mode
    return &CommandResult{Output:
        fmt.Sprintf("session permission mode set to %s\n",
            mode)}, nil
}

func (c *PermissionsCommand) addSession(pattern, action string,
    priority int) (*CommandResult, error) {
    if _, err := permissions.ParsePattern(pattern); err != nil {
        return nil, err
    }
    a, err := actionFromName(action)
    if err != nil {
        return nil, err
    }
}

```

```

_ = a
_ = priority
return &CommandResult{
    Output: fmt.Sprintf("session-only %s rule added:
        %s\n(use `helixcode permissions add` to persist)\n",
            action, pattern),
}, nil
}

func (c *PermissionsCommand) removeSession(pattern string)
(*CommandResult, error) {
return &CommandResult{
    Output: fmt.Sprintf("session-only rule removed: %s\n(use
        `helixcode permissions remove` to persist)\n", pattern),
}, nil
}

func defaultLoader(mode string) (*permissions.FileLoader, error)
{
home, err := os.UserHomeDir()
if err != nil {
    return nil, err
}
cwd, err := os.Getwd()
if err != nil {
    return nil, err
}
return &permissions.FileLoader{
    UserPath:    filepath.Join(home, ".helixcode",
        "permissions.yaml"),
    ProjectPath: filepath.Join(cwd, ".helixcode",
        "permissions.yaml"),
    Mode:        mode,
}, nil
}

func actionToName(a confirmation.Action) string {
switch a {
case confirmation.ActionAllow:
    return "allow"
case confirmation.ActionAsk:
    return "ask"
case confirmation.ActionDeny:
    return "deny"
}
return "unknown"
}

```

```
func actionFromName(s string) (confirmation.Action, error) {
    switch s {
    case "allow":
        return confirmation.ActionAllow, nil
    case "ask":
        return confirmation.ActionAsk, nil
    case "deny":
        return confirmation.ActionDeny, nil
    }
    return 0, fmt.Errorf("invalid action %q (allow|ask|deny)", s)
}
```

Register the command in the existing command-registry init function (search for where other commands are registered, e.g. `registry.Register(...)`). Add:

```
registry.Register(NewPermissionsCommand())
```



Step 5: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -race -run
    'TestPermissionsCommand' ./internal/commands/
```

Expected: PASS (4 tests).



Step 6: Run the whole commands package + permissions package

```
cd HelixCode && go test -count=1 -race ./internal/commands/... ./
    internal/tools/permissions/...
```

Expected: PASS.



Step 7: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
    placeholder" internal/commands/permissions_command.go &&
    echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 8: Commit

```
git -C HelixCode add internal/commands/permissions_command.go
    internal/commands/permissions_command_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F02-T11): /permissions slash command via internal/
    commands
```

Subactions: list (default), mode, add (session-only), remove (session-only). Persistent edits route to `helixcode permissions add`.

4 unit tests including unknown-mode rejection.

Co-Authored-By: Claude Opus 4.7 (1M context)

<noreply@anthropic.com>

EOF

)"

Task 12: Challenge with three runtime-evidence scenarios

Files: - Create: helix_code/tests/e2e/challenges/permissions/expected.json - Create: helix_code/tests/e2e/challenges/permissions/run.sh - Create: helix_code/tests/e2e/challenges/permissions/README.md



Step 1: Create the Challenge directory

```
mkdir -p helix_code/tests/e2e/challenges/permissions
```



Step 2: Write expected.json

Create helix_code/tests/e2e/challenges/permissions/expected.json:

```
{
  "name": "permissions/rule-engine-end-to-end",
  "feature": "P1-F02 - Permission Rule System",
  "scenarios": [
    {
      "id": "S1-read-auto-allowed-under-dontAsk",
      "input": {
        "permission_mode": "dontAsk",
        "tool": "Bash",
        "command": "ls -la /tmp"
      },
      "expected_decision": "allow",
      "expected_match_pattern_substring": "ls"
    },
    {
      "id": "S2-destructive-denied-under-default",
      "input": {
        "permission_mode": "default",
        "rules_yaml": "apiVersion: helixcode.permissions/v1\nrules:\n  - pattern: \"Bash(rm*)\"\n    action: deny\n    priority: 1000\n",

```

```

        "tool": "Bash",
        "command": "rm -rf /tmp/helixcode-challenge-marker"
    },
    "expected_decision": "deny",
    "expected_marker_path": "/tmp/helixcode-challenge-marker",
    "expected_marker_present_after": true
},
{
    "id": "S3-smuggle-via-cmd-substitution-denied",
    "input": {
        "permission_mode": "auto",
        "rules_yaml": "apiVersion: helixcode.permissions/
v1\nrules:\n  - pattern: \"Bash(rm*)\"\\n    action:
deny\n    priority: 1000\n",
        "tool": "Bash",
        "command": "echo hi $(rm -rf /tmp/helixcode-smuggle-
marker)"
    },
    "expected_decision": "deny",
    "expected_marker_path": "/tmp/helixcode-smuggle-marker",
    "expected_marker_present_after": true
}
]
}

```



Step 3: Write run.sh (the Challenge driver)

Create helix_code/tests/e2e/challenges/permissions/run.sh:

```

#!/usr/bin/env bash
# Challenge: P1-F02 – Permission Rule System end-to-end runtime
# evidence.
# Exits 0 only if all three scenarios produce the expected
# decisions
# AND the filesystem markers prove that denied commands really
# did NOT execute.
set -euo pipefail

HERE=$(cd "$(dirname "$0")" && pwd)
ROOT=$(cd "$HERE/../../../../.." && pwd)
BIN="$ROOT/bin/helixcode"

if [[ ! -x "$BIN" ]]; then
    echo "[setup] building helixcode binary..."
    (cd "$ROOT" && go build -o bin/helixcode ./cmd/cli)
fi

WORK=$(mktemp -d)

```

```

trap 'rm -rf "$WORK"' EXIT

# Scenario 1: read auto-allowed under dontAsk
echo "=== S1: read auto-allowed under dontAsk ==="
S1=$(("$BIN" permissions check Bash --command "ls -la /tmp" --
      permission-mode dontAsk)
echo "$S1"
if ! echo "$S1" | grep -q "decision: allow"; then
    echo "FAIL S1: expected decision: allow"
    exit 1
fi

# Scenario 2: destructive denied under default + explicit rule
echo
echo "=== S2: destructive denied under default ==="
MARKER1=/tmp/helixcode-challenge-marker
echo "present" > "$MARKER1"
RULES_DIR="$WORK/.helixcode"
mkdir -p "$RULES_DIR"
cat > "$RULES_DIR/permissions.yaml" <<'EOF'
apiVersion: helixcode.permissions/v1
mode: default
rules:
  - pattern: "Bash(rm*)"
    action: deny
    priority: 1000
EOF
S2=$(HOME="$WORK" "$BIN" permissions check Bash --command "rm
      -rf $MARKER1" --permission-mode default)
echo "$S2"
if ! echo "$S2" | grep -q "decision: deny"; then
    echo "FAIL S2: expected decision: deny"
    exit 1
fi
if [[ ! -e "$MARKER1" ]]; then
    echo "FAIL S2: marker $MARKER1 was deleted — deny rule did not
        block exec"
    exit 1
fi

# Scenario 3: smuggle via $() denied
echo
echo "=== S3: smuggle via command substitution denied ==="
MARKER2=/tmp/helixcode-smuggle-marker
echo "present" > "$MARKER2"
S3=$(HOME="$WORK" "$BIN" permissions check Bash --command "echo
      hi \$(rm -rf $MARKER2)" --permission-mode auto)
echo "$S3"

```



```

if ! echo "$S3" | grep -q "decision: deny"; then
    echo "FAIL S3: expected decision: deny (smuggled rm in \${})"
    exit 1
fi
if [[ ! -e "$MARKER2" ]]; then
    echo "FAIL S3: marker $MARKER2 was deleted — smuggle was not
        blocked"
    exit 1
fi

# Cleanup markers
rm -f "$MARKER1" "$MARKER2"

echo
echo "PASS: all three scenarios produced expected decisions and
    markers preserved"

```



Step 4: Make it executable

```
chmod +x helix_code/tests/e2e/challenges/permissions/run.sh
```



Step 5: Write README.md

Create helix_code/tests/e2e/challenges/permissions/README.md:

Challenge — Permission Rule System (P1-F02)

End-to-end runtime evidence that the permission engine's
decisions
correspond to real filesystem outcomes.

Scenarios

1. ****S1 — read auto-allowed under dontAsk****: ``ls -la /tmp``
resolves to ``allow``.
2. ****S2 — destructive denied under default****: a ``Bash(rm*) deny``
rule blocks
``rm -rf $MARKER``; the marker file is verifiably ****still
present**** after the call.
3. ****S3 — smuggle via \$() denied****: ``echo hi $(rm -rf $MARKER)``
resolves to ``deny``
even under ``--permission-mode auto``; the marker is verifiably
****still present****.

Run

```

```bash
cd HelixCode && tests/e2e/challenges/permissions/run.sh

```

Exit 0 means PASS. Exit non-zero means at least one scenario produced the wrong decision or the marker was tampered with.

## Mutation test (CONST-039)

To verify the Challenge actually catches a broken engine, temporarily change `internal/tools/permissions/rule_engine.go` to skip the deny aggregation:

```
// in aggregate(), before the switch:
hadDeny = false // <-- mutation
```

Re-run `run.sh`. It MUST FAIL on S2 or S3. **Revert** the mutation and confirm PASS.

- [ ] **\*\*Step 6: Run the Challenge end-to-end\*\***

```
```bash
cd HelixCode && tests/e2e/challenges/permissions/run.sh
```

Expected: prints S1/S2/S3 sections, ends with PASS:. Exit 0.



Step 7: Capture runtime evidence

```
cd HelixCode && tests/e2e/challenges/permissions/run.sh 2>&1 |
tee /tmp/p1-f02-t12-evidence.txt
```

Save the entire output — it goes verbatim into the close-out commit body in T13 (and into `06_phase_1_evidence.md`).



Step 8: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" tests/e2e/challenges/permissions/ && echo
"BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 9: Commit

```
git -C HelixCode add tests/e2e/challenges/permissions/
git commit -m "$(cat <<'EOF'
feat(P1-F02-T12): Challenge for permission rules with runtime
evidence"
```

Three scenarios driven by `helixcode permissions` check:

S1: `dontAsk auto-`allows `ls -la /tmp`.

S2: `explicit Bash(rm*) deny` blocks `rm` against a real `/tmp` marker;
marker still present after the call.

```
S3: command-substitution smuggle (echo hi $(rm ...)) denied
    under
    auto preset; marker still present.
```

Mutation-test recipe in README.md ensures the Challenge will FAIL
if
the engine's deny aggregation is broken.

Runtime evidence: see commit body of P1-F02-T13 close-out.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 13: Feature 2 close-out + push

Files: - Modify: docs/improvements/06_phase_1_evidence.md (paste runtime evidence) - Modify: docs/improvements/PROGRESS.md (flip F02 → F03 active)



Step 1: Append runtime evidence to evidence file

Append to docs/improvements/06_phase_1_evidence.md under the F02 §:

Task evidence trail

- T01 - ``<commit-sha-from-T01>`` - bootstrap evidence + advance PROGRESS
- T02 - ``<commit-sha-from-T02>`` - Wildcard field on confirmation.Condition (5 unit tests)
- T03 - ``<commit-sha-from-T03>`` - permissions package skeleton
- T04 - ``<commit-sha-from-T04>`` - shell_splitter via mvdan.cc/sh/v3 (10 unit tests)
- T05 - ``<commit-sha-from-T05>`` - RuleEngine with priority + aggregation (9 test groups)
- T06 - ``<commit-sha-from-T06>`` - five mode presets + command lists (8 unit tests)
- T07 - ``<commit-sha-from-T07>`` - YAML rule loader with project-over-user precedence (8 unit tests)
- T08 - ``<commit-sha-from-T08>`` - permissions.Engine facade + Policy registration (3 unit tests)
- T09 - ``<commit-sha-from-T09>`` - --permission-mode flag + integration tests (3 tests, NO mocks)
- T10 - ``<commit-sha-from-T10>`` - helixcode permissions {list,add,remove,check} (4 unit tests)
- T11 - ``<commit-sha-from-T11>`` - /permissions slash command (4 unit tests)

- T12 — ``<commit-sha-from-T12>`` — Challenge with runtime evidence

Challenge runtime evidence (from T12)

...

`<paste verbatim contents of /tmp/p1-f02-t12-evidence.txt here>`

...

Anti-bluff scan

...

```
$ cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/tools/permissions/ tests/e2e/
challenges/permissions/ tests/integration/permissions/
cmd/cli/permissions_cmd.go internal/commands/
permissions_command.go
```

clean

...

Verify-foundation gate

...

```
$ cd /run/media/milosvasic/DATA4TB/Projects/HelixCode && make
verify-foundation
```

`<paste output>`

...

Closure

F02 closed 2026-05-05. F03 (Tool Result Persistence) unblocked.

Replace `<commit-sha-from-TNN>` placeholders with the actual short SHAs of each task's commit:

```
# fill them in by copy/paste from `git log --oneline -15`
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode log --
oneline -15
```



Step 2: Run verify-foundation gate

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode && make
verify-foundation 2>&1 | tee /tmp/p1-f02-t13-verify.txt
```

Paste the output into the evidence file's "Verify-foundation gate" block. Expected: exit 0 (or warn-mode if the LLMsVerifier dual-pin parking-lot warning is still open — copy the warning into the evidence verbatim).



Step 3: Run final regression test

```
cd HelixCode && go test -count=1 -race ./internal/tools/
permissions/... ./internal/tools/confirmation/... ./
internal/commands/... ./cmd/cli/... && \
go test -count=1 -race -tags=integration ./tests/integration/
permissions/...
```

Expected: PASS everywhere.



Step 4: Anti-bluff smoke (final)

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/tools/permissions/ tests/e2e/challenges/permissions/ \
tests/integration/permissions/ cmd/cli/permissions_cmd.go \
internal/commands/permissions_command.go && echo "BLUFF FOUND"
|| echo "clean"
```

Expected: clean.



Step 5: Update PROGRESS.md

Edit docs/improvements/PROGRESS.md lines 10-18:

```
## Current focus
- **Active phase:** P1 – claude-code feature porting
- **Active feature:** F03 – Tool Result Persistence (awaits its
own writing-plans cycle)
- **Active task:** pending
- **Last completed:** P1-F02-T13 – Feature 2 (Permission Rule
System) close-out + push
- **Owner:** agent (Claude Opus 4.7)
- **Started:** 2026-05-04
- **Last touched:** 2026-05-05
- **Blocked-on:** none
```

Mark every F02 task [x] in the “Active feature task list (P1-F02 …)” block.

Add a Decision-log entry:

```
- 2026-05-05 – Feature 2 (Permission Rule System) closed.
Thirteen sub-commits; extended existing internal/tools/
confirmation.PolicyEngine + 5 claude-code modes as named
rule-presets composed with autonomy gradient. Smuggle-
resistant via mvdan.cc/sh/v3.
```



Step 6: Stage and commit close-out

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
    improvements/06_phase_1_evidence.md docs/improvements/
    PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
chore(P1-F02-T13): Feature 2 (Permission Rule System) close-out
```

Thirteen sub-commits. Extended internal/tools/
confirmation.PolicyEngine
with a Wildcard Condition field; new internal/tools/permissions
package
loads layered YAML rule files (~/.helixcode + project) and
produces a
Policy that delegates to a smuggle-resistant rule engine
(mvdan.cc/sh/v3 walker handles \$(...), backticks, heredocs,
quoted
operators, pipelines). Five claude-code mode presets (default |
auto |
acceptEdits | dontAsk | bypassPermissions) compose with the
existing
AutonomyMode gradient. Full CLI surface: --permission-mode flag,
helixcode permissions {list,add,remove,check} subcommands, and a
/permissions slash command via internal/commands.

Challenge runtime evidence (verbatim):
<paste S1/S2/S3 transcript from /tmp/p1-f02-t12-evidence.txt
here>

Anti-bluff scan: clean.
Verify-foundation gate: exit 0 (or LLMsVerifier-pin parking-lot
warning).

PROGRESS advanced: F02 done; F03 (Tool Result Persistence)
unblocked.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"



Step 7: Push to all configured remotes (no force, per CONST-043)

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode remote -v
```

For each remote, push without force:

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    origin main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    github main 2>/dev/null || true
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    gitlab main 2>/dev/null || true
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    upstream main 2>/dev/null || true
```

(If any remote name is not configured, the `|| true` suppresses the error. Only `origin` is required.)

Verify upstream parity:

```
for r in origin github gitlab upstream; do
    if git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode
        remote get-url "$r" >/dev/null 2>&1; then
        echo "=== $r ==="
        git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode ls-
            remote --heads "$r" main
    fi
done
```

Expected: all configured remotes show identical SHA = HEAD.

Self-review against the spec

Walked the spec section-by-section against the plan:

- **§1.4 S1 (make verify-compile exits 0)** — covered by T03 step 3, T08 step 5, T13 step 3.
- **§1.4 S2 (unit tests with -race)** — every TDD task uses `-race`.
- **§1.4 S3 (integration test, no mocks)** — T09.
- **§1.4 S4 (Challenge + runtime evidence pasted)** — T12 + T13.
- **§1.4 S5 (anti-bluff smoke clean)** — every task ends with the smoke check.
- **§1.4 S6 (CLI flag + subcommands work)** — T09 + T10.
- **§1.4 S7 (slash command registered)** — T11.
- **§2.3 component table** — every entry maps to T02-T11.
- **§3.1 YAML schema with apiVersion** — T07 enforces `helixcode.permissions/v1`; T07 step 1 includes a `TestLoad_UnknownAPIVersionIsError`.
- **§3.4 layer override + within-layer priority** — T07 covers project-over-user identical-pattern; T05 covers within-layer priority sort.
- **§4 mode presets** — T06.
- **§5 compound aggregation (deny > ask > allow)** — T05 has `TestEvaluate_CompoundAggregation_DenyWins` and `_AskOverAllow`.
- **§5 fail-closed on shell parse error** — T05 has `TestEvaluate_ShellParseErrorIsDeny`.
- **§7 file mode 0600 / dir mode 0700** — T07 has `TestSave_WritesFileWith0600Mode`.
- **§8.5 mutation test** — documented as a manual recipe in T09 step 7 and T12 README.

No spec section is uncovered.

Type consistency check: `Decision`, `Rule`, `RuleSet`, `FileLoader`, `PresetRules`, `ParsePattern`, `NewRuleEngine`, `NewEngine`, `Decide`, `IsValidMode`, `ValidModes`, `Scope` — all referenced consistently across tasks.

Placeholder scan: every step has either real code, a real command, or a real verification check. The only literal `<placeholder>` strings are the SHA fillers in T13 step 1 (intentional — they cannot be known until the prior commits land).

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-05-p1-f02-permission-rules.md`. Two execution options:

1. Subagent-Driven (recommended) — I dispatch a fresh subagent per task, review between tasks, fast iteration.

2. Inline Execution — Execute tasks in this session using `executing-plans`, batch execution with checkpoints.

Which approach?